LUNEX — VERSION 0.2 — DRAFT

LUNEX

A unified, object-oriented reference model for operational technology, safety, security, and AI.

lunex.cloud
Twenty-one sub-models · Full specification, with diagrams

LUNEX Specification

Version 0.2 — Draft A unified, object-oriented reference model for operational technology, safety, security, and AI

Status of this document

This is a **draft specification**, authored by a single contributor and not yet reviewed by an independent working group. It is published to invite scrutiny, not to claim authority. Every design decision in this document is falsifiable — if you find a case it doesn't handle, that is more valuable feedback than agreement. See the repository's `README.md` for how to contribute.

Table of Contents

1. Introduction
2. Relationship to Existing Standards
3. How to Read This Specification
4. Glossary
5. Sub-model 1 — Object / Class Model
6. Sub-model 2 — Asset Hierarchy / Namespace
7. Sub-model 3 — Topology Model
8. Sub-model 4 — Behavioral / State Model
9. Sub-model 5 — Recipe & Execution Model
10. Sub-model 6 — Safety
11. Sub-model 7 — Security Zones & Conduits
12. Sub-model 8 — Identity & Access Management
13. Sub-model 9 — Vulnerability & Patch Management
14. Sub-model 10 — Security Incident Response
15. Sub-model 11 — Data / AI Layer
16. Sub-model 12 — Naming & Terminology
17. Sub-model 13 — Management of Change
18. Sub-model 14 — Collective Status (Rollup)
19. Sub-model 15 — Alarm Management
20. Sub-model 16 — Alarm Response Guidance
21. Sub-model 17 — Scenario Simulation & Operator Guidance
22. Sub-model 18 — Situational Awareness
23. Sub-model 19 — Historian & Analytics
24. Sub-model 20 — Predictive Maintenance & Improvement
25. Sub-model 21 — Closed-Loop AI Control
26. References
27. Appendix A — Schema Reference

1. Introduction

LUNEX — *Layered Unified Namespace for Entities & eXtensibility*. Every object in the model, physical or not, is an **Entity** (`LunexObject`, Sub-model 1); the **Namespace** is the addressable path every one of them resolves to (Sub-model 2); **Layered** describes the five-band structure the twenty-one sub-models are organized into (§3); **eXtensibility** is why this version added five sub-models (Recipe & Execution, Management of Change, Identity & Access Management, Vulnerability & Patch Management, Security Incident Response) to the original sixteen without disturbing any of them, and why a twenty-second could be added the same way.

1.1 Why LUNEX exists

Operational technology has no single reference model that spans its full breadth. PackML describes machine state behavior. ISA-88 describes batch process structure. ISA-95 describes the enterprise-to-control hierarchy. IEC 61508/61511 describe functional safety. IEC 62443 describes industrial cybersecurity. ISA-18.2 describes alarm management. Each is authoritative within its own scope, and each was written before the others, largely independent of them. An engineer building a modern OT system today must mentally stitch these together — and none of them anticipated the AI/data layer that is now a first-class part of the system, not an add-on.

LUNEX is an attempt to provide that missing unification: **one object model**, from a single Sensor up to the AI system that predicts, recommends, and — with approval — acts on the process, with every layer built on the same base class and the same design discipline.

1.2 Intentions

- **Completeness over minimalism.** LUNEX intentionally covers more ground than PackML or ISA-88 alone — process control, safety, security, alarm management, and AI/data are treated as one connected model, not five separate concerns bolted together.
- **Reuse before invention.** Every sub-model was tested against one question before introducing new vocabulary or structure: *does an existing standard already define this correctly?* Where the answer was yes (SIL, SIF, Security Levels, ISA-18.2's alarm states, IEC 60204-1's stop categories), LUNEX reuses it unchanged. New vocabulary was introduced only where no adequate term existed, or where an existing term already carried a conflicting meaning within this model.
- **One base class, no exceptions.** Every object in LUNEX — a pressure sensor, a safety interlock, a security zone, an alarm, an AI-generated recommendation — is a `LunexObject`. This is not a simplification for its own sake; it means every later capability (addressing, telemetry, rollup status, situational awareness) applies uniformly to everything in the model, including capabilities that didn't exist when the object class was first defined.
- **AI as a structural citizen, not an add-on.** Digital twins, predictive maintenance, scenario simulation, and closed-loop AI control are modeled with the same rigor and the same base class as a pressure transmitter — including explicit governance (`operatingBounds`, `requiresApproval`, `manualOverrideAvailable`) that treats autonomous behavior as something to be bounded and audited, not merely enabled.
- **A living document.** LUNEX is versioned and expected to change. Sub-model 12 (Naming & Terminology) exists specifically to keep every naming and design decision traceable, so that future revisions can be judged against the same discipline this version was built with.

1.3 What LUNEX is not

LUNEX does not replace IEC 61508/61511, IEC 62443, ISA-18.2, or IEC 60204-1 — it *reuses* them. An organization implementing LUNEX still needs a SIL/SIF assessment performed to IEC 61508/61511, still needs an IEC 62443 zone-and-conduit segmentation exercise, and so on. LUNEX's contribution is the object model that lets the *outputs* of those exercises live inside one consistent, addressable, machine-readable structure — not a replacement for the domain expertise those standards require.

2. Relationship to Existing Standards

Standard	What it defines	How LUNEX relates to it
ISA-88 (S88)	Batch process control structure (Process Cell, Unit, Equipment Module); recipe levels and procedural steps (renamed and bound to the asset hierarchy — see Sub-model 5)	LUNEX's `Cell` (Sub-model 2) reuses S88's "Process Cell" concept directly, and `System` is the equivalent of S88's "Unit" — a functional grouping within a Cell, not a control system (DCS/SCADA/PLC). LUNEX's Assembly/Device/Component layers are a generalization of S88's structural pattern to non-batch processes. Sub-model 5 reuses S88's recipe levels (General/Site/Master/Control) and procedural hierarchy (Procedure/Unit Procedure/Operation/Phase) unchanged.
ISA-95 (S95)	Enterprise-to-control hierarchy (Enterprise, Site, Area, Work Center)	LUNEX's `Area` (Sub-model 2) is S95's term, reused unchanged. `Realm` / `Domain` / `Location` replace S95's `Enterprise` / `Division` / `Site` — broadened beyond a single industrial plant, and chosen to enable the addressable namespace in Sub-model 11 (see Sub-model 12 for the full reasoning).
PackML (ISA-TR88.00.02)	Machine state behavior for packaging equipment	LUNEX's Sub-model 4 state machine is a generalization of PackML's state pattern to any object class, not only packaging machines — including `RecipeExecution` (Sub-model 5), which reuses it directly rather than defining its own.
IEC 61508 / IEC 61511	Functional safety, Safety Integrity Level (SIL), Safety Instrumented Function (SIF)	Reused **unchanged**. Sub-model 6 makes `Interlock` a first-class object specifically so it can carry the proof-test history IEC 61508/61511 require. A SIF is modeled as its own physically independent Assembly, per IEC 61511's independence requirement, with `silTarget` / `silAchieved` split explicitly.
ISO 13849-1 / IEC 62061	Machine/discrete safety: Performance Level (PL), architecture Category, SIL for machinery — a parallel rating system to IEC 61508/61511, not equivalent to it	Reused **unchanged** as `Interlock.performanceLevel` / `category` (Sub-model 6), selected via `safetyStandard` rather than merged into the SIF's SIL fields.
IEC 61800-5-2	Safe-motion functions for adjustable-speed drives (STO, SS1, SS2, SOS, SLS, SDI, SLP)	Reused **unchanged** as `SafeMotionFunction` (Sub-model 6), attached to the Drive Device already established as LUNEX's canonical multi-interface example (Sub-model 1).
IEC 62443	Industrial cybersecurity: zones, conduits, Security Levels (SL); patch management (-2-3); incident response (-2-1); secure product development (-4-1); system security requirements (-3-3)	Reused **unchanged**. Sub-model 7 introduces `Zone` and `Conduit` as first-class objects carrying IEC 62443's SL-Target/SL-Achieved directly. Sub-models 8–10 (Identity & Access Management, Vulnerability & Patch Management, Security Incident Response) each reuse a different part of IEC 62443's scope that Sub-model 7 alone did not cover.
NIST Cybersecurity Framework (CSF)	Identify/Protect/Detect/Respond/Recover functions	Reused **unchanged** for `SecurityIncident` 's state machine (Sub-model 10): `Detected → Contained → Eradicated → Recovered → Closed` maps directly onto CSF's Respond/Recover functions.
CVE / CVSS	Vulnerability identification and scoring	Reused **unchanged** as `Vulnerability.cveRef` / `cvss` (Sub-model

		9).
ISA-18.2	Alarm management: priority, states, shelving, performance metrics	Reused **unchanged** for the concepts (priority, shelving, Return-to-Normal, performance metrics); Sub-model 15 defines the concrete `Alarm` object and its own state machine (deliberately not reusing Sub-model 4's, since operator-response states and object-behavior states answer different questions). Sub-model 19 computes the alarm performance metrics ISA-18.2 requires (alarms/hour, bad actors, time-to-acknowledge).
IEC 60204-1	Electrical safety of machinery, stop categories 0/1/2	Reused **unchanged**. Sub-model 6 confirms LUNEX's Emergency/Quick/Controlled Stopping states map directly to Category 0/1/2, including `SafeMotionFunction.stoCategory`.
Purdue Enterprise Reference Architecture (PERA)	Layered model of enterprise/control network levels (L0–L4)	Sub-model 2's hierarchy is mapped onto Purdue levels at every rung, and Sub-model 7's security Zones commonly (not necessarily) align with a Purdue band.
UCUM	Unified Code for Units of Measure	Reused **unchanged** as the required encoding for every `hasUnit` value (Sub-model 12).
EU Cyber Resilience Act	Vulnerability handling and SBOM obligations for connected products (applicable from 2027)	Not a structural dependency, but the regulatory driver behind Sub-model 9's `Vulnerability` / `PatchRecord` objects existing at all.
OPC UA	Industrial interoperability protocol and information modeling	Not a structural dependency of LUNEX, but a natural transport/encoding candidate for the Sub-model 11 Telemetry Envelope in a real implementation. LUNEX does not mandate OPC UA; it deliberately stays protocol-agnostic.

3. How to Read This Specification

Each sub-model chapter (Sections 5–25) follows the same structure:

- **Purpose** — the functional problem this sub-model solves, in plain language, and why it couldn't be solved by an earlier sub-model alone.
- **Technical Description** — the formal object definitions: classes, attributes, state machines, and relationships to other sub-models.
- **Application Guidance** — how an implementer actually uses this sub-model: what to configure, what to compute, and a worked example.

The twenty-one sub-models are organized into five bands, mirroring the grouping used throughout the project's diagrams and website:

Band	Sub-models	What it answers
Foundation	1–5	What is an object, where does it live, how is it wired, how does it behave, and what procedure is it executing?
Safety & Security	6–10	Interlocks/SIFs, segmentation, identity, vulnerability/patch posture, and incident response — all depend on 1–5, independent of each other.
Cross-cutting	11–13	Data/AI, naming discipline, and governed change — independent of each other, all depending on 1–5 (and, for 13, able to reference any object in 6–10 as the subject of a change).
Operator capability	14–18	Depend on 11 and on each other in combination: rollup, alarm management, alarm response, scenario simulation, situational awareness.
Autonomous layer	19–21	Depend on 11 and, for 20–21, on 14–17: historian/analytics, predictive maintenance, closed-loop AI control.

Sub-models build on each other in the order presented within each band, and bands build on earlier bands — nothing in a later band is assumed by an earlier one.

The corresponding SVG diagram for each sub-model is referenced by filename; this document is the narrative and technical companion to those diagrams, not a replacement for them.

4. Glossary

Terms are listed alphabetically. Terms reused unchanged from an external standard are marked with that standard in brackets.

AccessGrant — The binding between an `Identity` and a `Role`, scoped to a specific object and optionally time-bounded (Sub-model 8). A `Role` is a reusable template; an `AccessGrant` is where it actually applies.

actionable — A derived boolean on `Alarm` (Sub-model 15): whether the operator can still change the outcome. Automatically computed from the linked Interlock's state — `true` while the Interlock is in Standby (not yet tripped), `false` once it has tripped.

Actuator — One of the five universal device classes (Sub-model 1); converts a control signal into physical action (valve, motor, heater, pump).

Alarm — A first-class `LunexObject` (Sub-model 15) representing a condition requiring operator attention. Priority is computed as severity × actionability, not severity alone.

AlarmResponseProcedure (ARP) — A reusable template (Sub-model 16), tied to an alarm type rather than an individual alarm instance, providing probable cause, consequence, corrective action, escalation, and a reference document. Mandatory for Priority 1/2 alarms.

Alarm.origin — Attribute on `Alarm` (Sub-model 20): `real-time | predictive`. Named `origin` rather than `source` specifically to avoid colliding with `Alarm.source` (Sub-model 15), which means "which Device."

Approval — One entry in a `ChangeRequest.approvals` list (Sub-model 13): a Role, the Identity that approved or rejected, a comment, and a timestamp.

architectureCategory — Attribute on `Interlock` (Sub-model 6): `B | 1 | 2 | 3 | 4`, the ISO 13849-1 architecture category — named `architectureCategory`, not `category`, because "Category" already means Stop Category 0–2 (IEC 60204-1) elsewhere in the same sub-model. Set only when `safetyStandard: ISO13849-PL`.

Area — A level in the Sub-model 2 hierarchy, reused directly from ISA-95. A process area within a Location.

Assembly — A level in the Sub-model 2 hierarchy: a topology pattern (see Sub-model 3) composed of Devices.

AIControlUnit — A `ControlUnit` subclass (Sub-model 21) representing an AI model that actively adjusts process setpoints. Occupies an ordinary ControlUnit slot in an Assembly; no new topology required.

automaticMode — Field on `SimulationPolicy` (Sub-model 17) and `RetentionPolicy` (Sub-model 19): `always | risk-based | none`. Paired with the universal `manualOverrideAvailable`.

bubbling — The Sub-model 14 principle that a parent object computes its Rollup from its direct children's Rollups only, never by scanning the full tree beneath it.

Cell — A level in the Sub-model 2 hierarchy, reused from ISA-88's "Process Cell."

ChangePolicy — A first-class `LunexObject` (Sub-model 13), named `ChangePolicy` rather than `Policy` because `SimulationPolicy` and `RetentionPolicy` already use "Policy" for a different kind of object (automatic-behavior settings): a standing rule — which `Role`s must approve, how many, and the minimum risk `Tier` — that a matching `ChangeRequest` resolves its `requiredApprovals` / `riskAssessment` against instead of each one being configured from scratch. Can only raise a `ChangeRequest`'s approval bar, never lower it.

ChangeRequest — A first-class `LunexObject` (Sub-model 13) generalizing the "someone must approve this" pattern found across `ImprovementRecommendation`, Safety/Security configuration, `AccessGrant` escalation, and patching into one governed, audited workflow. Its own value space (`Draft` through `Closed`) is deliberately separate from Sub-model 4's object-behavior states. Resolves its `requiredApprovals` / `riskAssessment` against a matching `ChangePolicy` when one exists.

Class — Attribute on `LunexObject`: a reference to the object's defining template in the organization's Inventory (Sub-model 1 §5.3), not a free-text string. Two objects share a `class` when instantiated from the same Inventory entry.

Cloud & Analytics — A capability flag in Sub-model 3: whether an Assembly's ControlUnit(s) stream telemetry to the AI/Analytics layer (Sub-model 11). Independent of wiring shape.

Component — Abstract class (Sub-model 1) for parts of a Device that are not independently addressable (e.g., a digital input card).

Conduit — A first-class `LunexObject` (Sub-model 7), reusing IEC 62443: the governed communication pathway between two Zones, where a Firewall/IDS ControlUnit typically sits.

Context Layer — Semantic relationships (`measures`, `partOf`, `hasUnit`, `semanticTag`) between objects (Sub-model 11), separate from and richer than the containment chain in Sub-model 2.

contributors — Field on a `Rollup` (Sub-model 14): the list of specific descendant objects responsible for a non-nominal severity tier.

ControlUnit — One of the five universal device classes (Sub-model 1); performs logic/decision-making (PLC, DCS, microcontroller, and by extension `AIControlUnit`).

coordinates — Optional attribute on `LunexObject` (Sub-model 1): `{ lat, lon } | null`. On the base class, not only on `Location`, since individual objects within one Location can be genuinely distributed (a pipeline's valves, a wind farm's turbines). Omitting it is a safe default, the same as omitting `physicalRef` — not an assertion of anything.

Device — Abstract class (Sub-model 1) for any independently addressable, assembly-level unit; composes zero or more Components. Optionally carries `physicalRef` when its physical hardware is shared with other functionally distinct Devices.

Digital Twin — The AI-side, queryable mirror of a physical object (Sub-model 11), kept in sync via the Telemetry Envelope. Holds only the *current* state — see Historian for historical state.

Domain — A level in the Sub-model 2 hierarchy: an optional business-unit grouping between Realm and Location.

Envelope — See Telemetry Envelope.

GuidanceRecommendation — A first-class `LunexObject` (Sub-model 17) holding a set of `ScenarioResult`s. `source` (later extended with a third value in Sub-model 20) indicates whether the guidance is `procedural`, `simulated`, or `predictive`. `state: Open | Applied | Dismissed | Expired` — overrides `methods()` with `apply()` / `dismiss()`; expires automatically as the process state it was generated against moves on.

goldenScenarioId — Field on `GuidanceRecommendation`: the id of the recommended `ScenarioResult`, or `null` if the scenarios are equivalent and no single best option is being asserted.

Health — Type of `LunexObject.health` (Sub-model 1): `{ score : 0-100, flags : string[] }`. Diagnostic condition, deliberately independent of `state` — an object can be `On` and simultaneously report degraded health.

Historian — An append-only record (Sub-model 19) of every Telemetry Envelope ever emitted, alongside (not gating) the Digital Twin. Also stores `VersionRecord` configuration baselines (Sub-model 13), reusing the same append-only principle.

HistorianRecord — One row in the Historian: path, timestamp, state, health, properties, and a reference to the `RetentionPolicy` governing its resolution.

High-Availability (HA) — A capability flag in Sub-model 3: whether an Assembly's ControlUnit(s) are deployed as an active/standby redundant pair. Independent of wiring shape.

id — Attribute on `LunexObject` (Sub-model 1): the system-generated, permanent identifier. Never reused, never renamed. Every `Ref` field in this specification (`DeviceRef`, `AlarmRef`, `ZoneRef`, etc.) points to an `id`, not a `tag`.

Identity — A first-class `LunexObject` (Sub-model 8) representing a human or non-human actor capable of acting on the model — broader than `Operator`, which deliberately records only job title and capacity, not permissions.

Inhibited — A Sub-model 4 state: Tier 1 (Restricted) per Sub-model 14. Reached from Standby (via Inhibiting) or from On (via Emergency Stopping). Requires an explicit Clear to leave.

ImprovementRecommendation — A first-class `LunexObject` (Sub-model 20) suggesting a configuration change based on a Historian pattern. `requiresApproval` is always `true` — no confidence threshold ever skips human review; approval itself is routed through a `ChangeRequest` (Sub-model 13).

Interlock — A first-class `LunexObject` (Sub-model 6), reused conceptually from IEC 61508/61511: a safety condition that forces or blocks a transition on a target Device. Carries its own proof-test history, plus `safetyStandard` / `performanceLevel` / `architectureCategory` for machine-safety (ISO 13849-1/IEC 62061) Interlocks.

Inventory — The Sub-model 1 term for a reusable class/template (chosen over "Library" for plainer, more approachable naming).

jobTitle — Attribute on `Operator` (Sub-model 1): a free-text descriptive string (e.g. "Control Room Operator"). Not a `Role` (Sub-model 8) and not a permissions grant — named `jobTitle`, not `role`, so it can't be mistaken for the `Role` class.

jumpToWorst() — A navigation function (Sub-model 18): from any hierarchy node, follows the Rollup's own `contributors` path directly to the Device/Alarm causing the worst status, bypassing manual layer-by-layer navigation.

Locked — A Sub-model 4 state: Tier 1 (Restricted). Reached from Off (via Locking), or from Standby/Inhibited (via Disabling → Off → Locking). The target of `lockType: lock`, and of `lockType: inhibit` when the object is already Off (since Inhibited is unreachable from Off).

lockType — Attribute on `Interlock` (Sub-model 6): `inhibit | lock`. Determines which Sub-model 4 transition the Interlock forces, given the object's current state.

Location — A level in the Sub-model 2 hierarchy: a plant, building, vessel, or remote site.

LunexObject — The abstract base class (Sub-model 1) every object in LUNEX inherits from, without exception.

Management of Change (MOC) — See `ChangeRequest`.

manualOverrideAvailable — Field on `SimulationPolicy` and `RetentionPolicy`: always `true`, regardless of `automaticMode`. Manual triggering is universal, not exclusive to a specific mode.

methods() — Attribute on `LunexObject` (Sub-model 1 §5.2.1): a computed view, not a stored list — returns the Sub-model 4 transition-triggers valid from the object's current `state`, excluding `Complete`. Empty for any transient state. `Alarm`, `Interlock`, `GuidanceRecommendation`, `PredictedEvent`, `ImprovementRecommendation`, `ChangeRequest`, and `SecurityIncident` override it with their own trigger sets; `RecipeExecution` does not.

operatingBounds — Attribute on `AIControlUnit` (Sub-model 21): required whenever `target` is the physical Device; may be empty only when `target` is the Digital Twin, since an unbounded action there cannot cause real-world harm.

Operator — A first-class `LunexObject` peer (Sub-model 1): `jobTitle : string`. Deliberately minimal — records identity and capacity, not permissions; resolves `OperatorRef` wherever it's used elsewhere in the model. See `Identity` (Sub-model 8) for the broader permissions-capable actor.

owner — Optional attribute on `LunexObject` (Sub-model 1): `RoleRef | null`, the `Role` (Sub-model 8) accountable for the object. `null` by default — accountability otherwise follows `parent` by convention. Not a permissions grant (see `AccessGrant`) and not a workflow rule (see `ChangePolicy`, Sub-model 13) — it only answers who is accountable.

parent — Attribute on `LunexObject` (Sub-model 1): the immediate container in the Sub-model 2 containment chain. Distinct from `physicalRef` (shared hardware) and from any Sub-model 4 transition relationship — three separate kinds of "points to another object."

PatchRecord — The record (Sub-model 9) of a firmware/software version applied to a Device, produced by a `ChangeRequest` reaching `Implemented`.

performanceLevel — Attribute on `Interlock` (Sub-model 6): `a | b | c | d | e`, the ISO 13849-1 Performance Level. Set only when `safetyStandard: ISO13849-PL` — never alongside a SIF's `silTarget` / `silAchieved` on the same Interlock.

Permission — An action (`view | operate | configure | approve`) scoped to an object or an entire Inventory class (Sub-model 8), held by a `Role`.

pfdAvg / pfh — Fields on a SIF Assembly (Sub-model 6): the calculated probability of failure on demand (low-demand mode) or probability of failure per hour (high-demand/continuous mode), feeding `silAchieved`. Mutually exclusive.

PredictedEvent — A first-class `LunexObject` (Sub-model 20) representing a forward-looking claim, either `internal-pattern` (Historian-derived, predictive maintenance) or `external-forecast` (e.g. weather, energy price — predictive analytics more broadly), distinguished by `basisType`. Raises into the ordinary Alarm system, tagged `PREDICTIVE`, rather than occupying a separate screen.

physicalRef — Optional attribute on `Device` (Sub-model 1): `DeviceRef | ComponentRef`, points to the actual shared, serviceable unit — a Device for simple hardware, or a specific Component (e.g. one CPU in a multi-CPU rack) when the Device's internal structure means different functional roles are backed by different sub-parts. Also confirms Sub-model 3's Integrated topology when shared across an entire chain.

priority — Attribute on `Alarm` (Sub-model 15): 1 (Critical) to 4 (Log Only), computed as severity × actionability.

properties — Attribute on `LunexObject` (Sub-model 1): `Map<Key, Value>` — the object's own data, including the current output of any continuous operational functionality (Sub-model 1 §5.2.2). Specific to the object's `class`; not standardized in shape beyond the map itself, though every value's unit must be a valid UCUM code (Sub-model 12).

Realm — The root level of the Sub-model 2 hierarchy: the whole organization's namespace.

Recipe — A template (Sub-model 5), not a running instance, with four refinement levels (`recipeLevel: product | location | equipment | execution`, each pointing to its parent via `derivedFrom`). See `RecipeExecution` for the instance.

RecipeElement — A step in a `Recipe`'s structure (Sub-model 5), named by the asset level that executes it: `runsOn: area | cell | system | assembly`. Elements nest downward (a child runs on a lower level than its parent), carry `childOrder: sequence | parallel`, and never extend to Device level — a Device has its own Sub-model 4 state.

RecipeExecution — A first-class `LunexObject` (Sub-model 5): one execution of a `Recipe` against a piece of equipment. Reuses the Sub-model 4 state machine directly, with no override of `methods()` — its execution is object behavior, not a different kind of question.

requiredApprovals — Field on `ChangeRequest` (Sub-model 13): the `Role`s (Sub-model 8) that must approve before the change can proceed, configurable per `changeType` or resolved automatically from a matching `ChangePolicy`.

requiresApproval — Attribute on `ImprovementRecommendation`: fixed to `true`. Has no false case, even at maximum confidence.

resetMode — Attribute on `Alarm` (Sub-model 15): `auto | manual`. Determines whether an alarm returns to Normal automatically once its condition clears, or requires an explicit operator reset via the Cleared (RTN) state.

riskThreshold — Field on `SimulationPolicy` / `RetentionPolicy` used when `automaticMode: risk-based`: a Sub-model 14 Tier value above which the automatic behavior triggers.

Role — A reusable set of `Permission`s (Sub-model 8), bound to a specific Identity and scope via an `AccessGrant`. Referenced by `ChangeRequest.requiredApprovals`.

Rollup — A computed view (Sub-model 14), not a stored class: `worstTier`, `tierCounts`, and `contributors` summarizing a node's own state plus its children's Rollups.

safetyStandard — Attribute on `Interlock` (Sub-model 6): `IEC61508-SIL | ISO13849-PL | IEC62061-SIL` — which rating system this Interlock was engineered against. Determines whether `performanceLevel` / `architectureCategory` or the SIF's `silTarget` / `silAchieved` applies.

SafeMotionFunction — A property set (Sub-model 6) on a Drive Device, reusing IEC 61800-5-2 unchanged: which safe-motion functions (STO, SS1, SS2, SOS, SLS, SDI, SLP) it supports.

ScenarioResult — A single simulated outcome (Sub-model 17) within a `GuidanceRecommendation`: description, predicted outcome, success probability, and whether it is the golden scenario.

SecurityIncident — A first-class `LunexObject` (Sub-model 10), peer of `Alarm` but not a subtype of it: a cybersecurity event with its own `Detected → Contained → Eradicated → Recovered → Closed` lifecycle [NIST CSF], distinct from ISA-18.2's process-alarm workflow.

Sensor — One of the five universal device classes (Sub-model 1); converts a physical quantity into a raw signal.

severity tier — See Tier.

shelvedUntil — Attribute on `Alarm`: the timestamp a Shelved alarm automatically returns to Unacknowledged. Shelving always expires; it cannot permanently silence an alarm.

SignalConverter — One of the five universal device classes (Sub-model 1); converts a control signal into a physical/power signal for the Actuator side.

SIF (Safety Instrumented Function) [IEC 61511] — Reused unchanged. Modeled in Sub-model 6 as an Assembly with `independent: true`, physically separate from the BPCS Assembly. Carries `silTarget` / `silAchieved`, not a single `SIL` field.

SIL (Safety Integrity Level) [IEC 61508] — Reused unchanged, split into `silTarget` / `silAchieved` on a SIF Assembly (Sub-model 6) — the same target/achieved split already used for Security Level.

silTarget / silAchieved — Fields on a SIF Assembly (Sub-model 6): the engineered SIL goal versus the SIL actually verified via `pfdAvg` / `pfh`. Mirrors `securityLevelTarget` / `securityLevelAchieved` (Sub-model 7).

SimulationPolicy — Governs when normal operation (not just alarms) triggers scenario simulation (Sub-model 17). See `automaticMode`, `riskThreshold`, `manualOverrideAvailable`.

SL (Security Level) [IEC 62443] — Reused unchanged, as `securityLevelTarget` / `securityLevelAchieved` on a `Zone` or `Conduit`.

Standby — A Sub-model 4 state: Tier 3 (Nominal) per Sub-model 14. Never a target of an Interlock.

State — The 15-value enumeration defined in Sub-model 4 (5 stable, 10 transient), shared by every `LunexObject` unless the object's class doesn't need a given branch. Reused unchanged by `RecipeExecution` (Sub-model 5).

System — A level in the Sub-model 2 hierarchy: a functional unit within a Cell, equivalent to ISA-88's Unit. Not a control system (DCS/SCADA/PLC) — that would be a `ControlUnit` (Sub-model 1) instance living inside this level.

tag — Attribute on `LunexObject` (Sub-model 1): the human/engineering-facing label (e.g. `PT-101`). Unlike `id`, `tag` can be renamed during the object's life without breaking any reference, since references use `id`.

target — Attribute name reused across two classes with different meanings, the same way `state` is: on `Interlock` (Sub-model 6), `target : DeviceRef` is the Device the Interlock forces or blocks a transition on; on `AIControlUnit` (Sub-model 21), `target : PhysicalDevice | DigitalTwin` is what its output is routed to. No object is ever both classes at once, so the two meanings never collide in practice — documented here rather than resolved by renaming, consistent with how `Alarm.state`'s own value space coexists with `Interlock.state`'s reuse of the Sub-model 4 enum.

Telemetry Envelope — The addressed, timestamped wrapper (Sub-model 11) around what `LunexObject` already exposes (class, state, health, properties). Streams to both the Digital Twin and the Historian.

Tier — The severity ranking defined in Sub-model 14: 0 (Critical) to 3 (Nominal), ranking the Sub-model 4 states by severity — a ranking the state machine itself does not carry. Reused directly as `ChangeRequest.riskAssessment` (Sub-model 13) and `SecurityIncident.severity` (Sub-model 10).

Transducer — One of the five universal device classes (Sub-model 1); converts a raw sensor signal into a usable signal.

tierCounts — Field on a `Rollup`: a count of descendants at each Tier, preserving multi-condition situations a single worst-case value would hide.

UCUM (Unified Code for Units of Measure) — Reused unchanged (Sub-model 12) as the required encoding for every `hasUnit` Context Layer value and every unit carried in `properties`.

VersionRecord — Produced automatically when a `ChangeRequest` (Sub-model 13) reaches `Implemented`: a before/after configuration snapshot, closing the gap a `ChangeRequest` alone leaves — that a change happened, but not what existed immediately before it.

votingArchitecture — Mandatory attribute on a SIF Assembly (Sub-model 6): `MooN` per layer (sensor, logicSolver, finalElement), reused unchanged from IEC 61508. Mandatory, unlike `physicalRef`, because an implicit default here would silently assert a SIL-relevant fact.

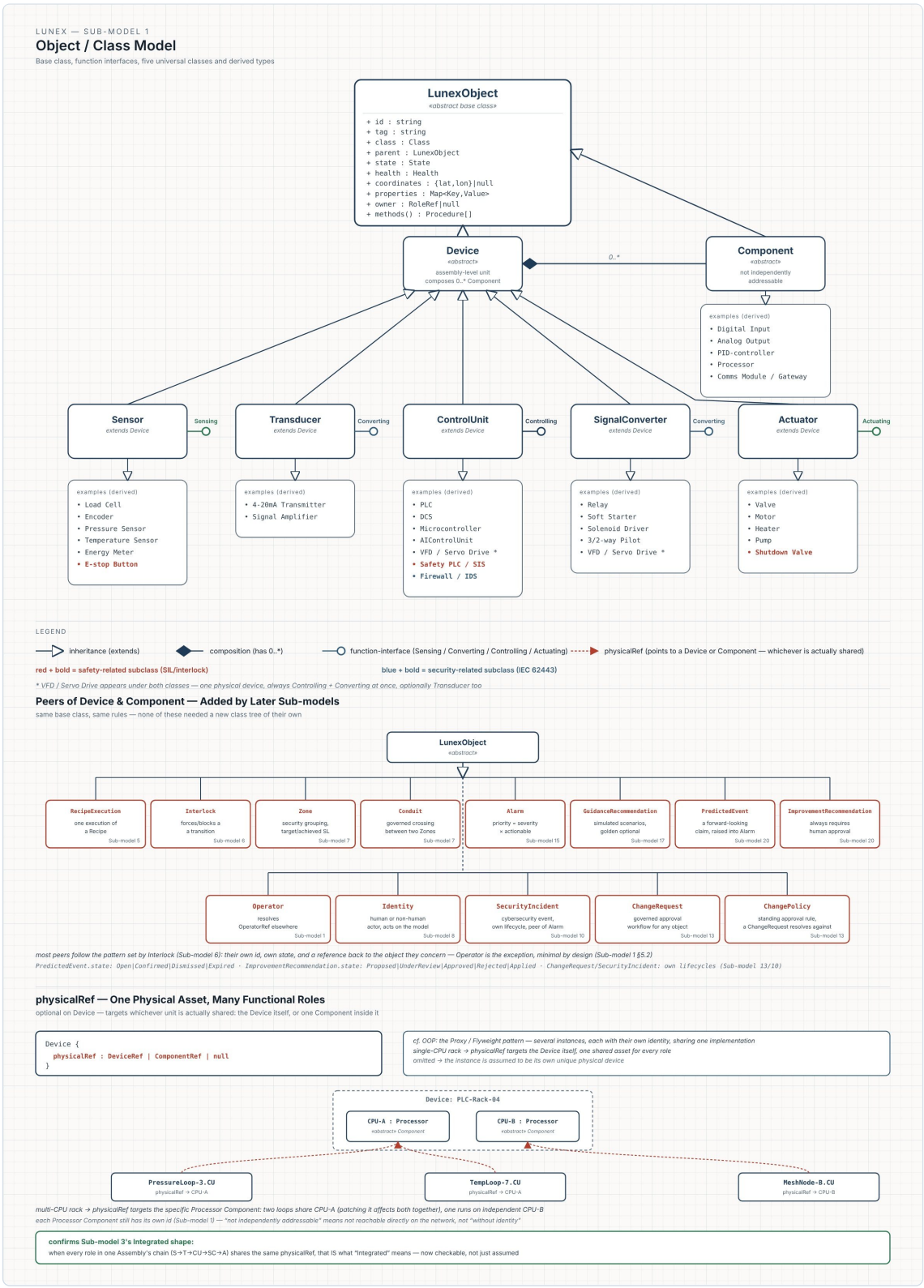
Vulnerability — A known weakness (Sub-model 9) tracked per Device via `cveRef` / `cvss`, with `state: open | mitigated | patched | accepted-risk`. `accepted-risk` is a legitimate permanent end state, not a default.

worstTier — Field on a `Rollup`: the lowest (worst) Tier present anywhere beneath a node.

Zone — A first-class `LunexObject` (Sub-model 7), reusing IEC 62443: a security grouping with a target and achieved Security Level. Not to be confused with the Sub-model 2 hierarchy, which uses `Area` instead specifically to keep this meaning unambiguous.

5. Sub-model 1 — Object / Class Model

Diagram: `lunex-object-model.svg`



lunex-object-model.svg

5.1 Purpose

Every OT reference model needs a base unit of "thing." PackML and ISA-88 each define their own, scoped to their own domain (a packaging machine state; a batch equipment module). LUNEX needs one that scales from a single pressure transmitter to an AI recommendation engine, without a different modeling approach at each scale. Sub-model 1 defines that base unit once, and everything built afterward — safety, security, alarms, AI — is a subclass or a peer of it, never a parallel system.

5.2 Technical Description

`LunexObject` (abstract base class). Every object in LUNEX, without exception, has:

```
LunexObject {
  id            : string
  tag           : string
  class         : Class
  parent        : LunexObject
  state         : State                 (Sub-model 4)
  health        : Health
  coordinates   : { lat, lon } | null
  properties    : Map<Key, Value>
  owner         : RoleRef | null         (Sub-model 8)
  methods()     : Procedure[]
}
```

- **`id`** — the system-generated, permanent identifier. Never reused, never renamed. This is what every `Ref` field elsewhere in this specification (`DeviceRef`, `AlarmRef`, `ZoneRef`, and so on) actually points to.
- **`tag`** — the human/engineering-facing label (e.g. `PT-101`, `PressureLoop-3.CU`). Unlike `id`, `tag` can be renamed during the object's life (a re-tagging project, a loop renumbering) without breaking any reference, because references use `id`.
- **`class`** — a reference to the object's defining template in the organization's Inventory (Sub-model 1 §5.3), not a free-text string. Two objects share a `class` when they were instantiated from the same Inventory entry.
- **`parent`** — the immediate container in the Sub-model 2 containment chain (e.g. a `Device`'s `parent` is its `Assembly`). This is structural containment, not `physicalRef` (Sub-model 1's optional shared-hardware pointer) and not the Sub-model 4 state machine's own transition graph — three separate relationships that happen to all involve one object pointing at another.
- **`health : Health`** — `{ score : 0-100, flags : string[] }`. Diagnostic condition, deliberately independent of `state`: an object can be `On` (Sub-model 4) and simultaneously report degraded `health` (e.g. a sensor drifting out of calibration) without that affecting which state-transitions are available.
- **`coordinates`** — optional geographic position, `{ lat, lon } | null`. Deliberately on `LunexObject` itself rather than only on `Location` (Sub-model 2): a Location's coordinates cover "where the site is," but individual objects within one Location can be genuinely kilometers apart — a pipeline's valves, a wind farm's turbines, a solar array's inverters. Omitting `coordinates` has the same safe default as omitting `physicalRef`: the object is simply not geo-located, not an assertion of anything. Most objects will never set this — it exists for the minority that are usefully mapped individually rather than as one dot per site.
- **`properties`** — the object's own data: current value, unit, setpoint, whatever is specific to its `class`. This is also where continuous operational output lives (see §5.2.1 below).
- **`owner`** — optional pointer to the `Role` (Sub-model 8) accountable for this object, not the person who happens to be acting on it right now (that's `Identity` / `Operator`, resolved per-action). `null` by default — most objects don't need one explicitly and inherit accountability from their `parent` by convention. Set it where accountability doesn't follow containment cleanly (a Zone spanning two Cells, a Recipe owned by a process engineering team rather than the Area it happens to run in) or where a `ChangeRequest` / `ChangePolicy` (Sub-model 13) needs to resolve "who is accountable for this" without a human having to look it up. This is deliberately the only governance field on the base class — *who may act* stays in Sub-model 8 (`AccessGrant`), *what the rules are* stays in Sub-model 13 (`ChangePolicy`); `owner` only answers "who is accountable," and does so for every object the same way `health` does for diagnostic condition.
- **`methods()`** — see §5.2.1.

`state` and `health` are deliberately separate: `state` is the Sub-model 4 operating mode; `health` is diagnostic condition, independent of mode.

5.2.1 `methods()` — Derived from State, Not a Free List

`methods()` is not an arbitrary, per-class method table. It is a **computed view**: the set of Sub-model 4 transition-triggers valid from the object's *current* `state`, minus `Complete` (which is an automatic, system-internal transition — never something a caller invokes). Calling `methods()` on an object in `Standby` and the same object in `On` returns two different, non-overlapping lists, because the underlying state machine (Sub-model 4) only exposes different outgoing edges from each.

Canonical trigger names, taken directly from the Sub-model 4 transition labels already established:

From state	`methods()` returns
`Locked`	`unlock()`
`Off`	`lock()`, `enable()`
`Standby`	`start()`, `disable()`, `inhibit()`
`Inhibited`	`clear()`, `disable()`
`On`	`controlledStop()`, `quickStop()`, `emergencyStop()`
any transient state (`Starting`, `Enabling`, `Disabling`, `Unlocking`, `Locking`, `Inhibiting`, `Clearing`, all three Stopping variants)	*(empty)*

The empty case for transient states is not an oversight — it follows directly from Sub-model 4's own graph, which was already drawn this way: while an object is completing its own transition, it accepts no new command until that transition reaches a stable state. This specification simply makes the rule explicit rather than leaving it implicit in the diagram.

Classes with their own state machine override `methods()` with their own trigger set, rather than inheriting the Sub-model 4 list:

Class	Sub-model	Override triggers
`Alarm`	15	`acknowledge()`, `shelve()`, `operatorReset()`, `suppress()` / `unsuppress()`, `takeOutOfService()` / `returnToService()`
`Interlock`	6	`clear()` (releases a tripped condition once the underlying cause is gone; distinct from the target Device's own `clear()`)
`GuidanceRecommendation`	17	`apply()` (Open → Applied), `dismiss()` (Open → Dismissed). `Expired` happens automatically as the underlying process state moves on, never via a method call.
`PredictedEvent`	20	`confirm()` (Open → Confirmed), `dismiss()` (Open → Dismissed). `Expired` is never a method call — it happens automatically once `predictedWindow` passes with no confirmation.
`ImprovementRecommendation`	20	`startReview()` (Proposed → UnderReview), `approve()` / `reject()` (UnderReview → Approved / Rejected), `apply()` (Approved → Applied). No method ever skips `UnderReview` — see Sub-model 20 §24.2, `requiresApproval`.
`AIControlUnit`	21	inherits the Sub-model 4 list unchanged — `enable()` / `disable()` are exactly `enable()` / `disable()` from the table above, since an AIControlUnit's own On/Off/Standby is the ordinary state machine, not a new one (Sub-model 21 §25.2)
`RecipeExecution`	5	*(no override — reuses the standard Sub-model 4 list; see Sub-model 5 §9.2)*
`ChangeRequest`	13	`submit()`, `approve()` / `reject()`, `implement()`, `verify()`, `close()` — see Sub-model 13 §17.2
`SecurityIncident`	10	`detect()`, `contain()`, `eradicate()`, `recover()`, `close()` — see Sub-model 10 §14.2

5.2.2 Scope Boundary: Continuous Operational Functionality

`methods()` covers discrete, externally-triggered state transitions. It deliberately does **not** cover continuous operational functionality — a PID loop recalculating every scan cycle, a Sensor sampling continuously, an AIControlUnit's own control algorithm running while it is `On`. This is not a gap; it is the same boundary this specification draws everywhere else: Sub-model 6 never specifies *how* a SIF computes its trip logic, only that it must; Sub-model 21 never specifies *which* algorithm an AIControlUnit runs, only its `objective` (intent, not implementation) and the governance around it. LUNEX is a structural and governance model, not a control-algorithm standard.

What continuous functionality needs is already covered, without any new mechanism:

Need	Already covered by
The current output of that continuous functionality	`properties` (this section)
Getting that output off the object	Telemetry Envelope (Sub-model 11)
Whether it's allowed to run at all	`state` + Interlock (Sub-model 6) + `operatingBounds` (Sub-model 21)
What it's trying to achieve, without prescribing how	`objective` (Sub-model 21, applicable to any ControlUnit, not only `AIControlUnit`)

`Device` (abstract, extends `LunexObject`) — an independently addressable, assembly-level unit. Composes zero or more `Component`.

```
Device {
  physicalRef : DeviceRef | ComponentRef | null
}
```

`physicalRef` is optional — most Devices don't need it, and omitting it means the instance is assumed to be its own unique physical asset. It exists for the common case where one physical unit plays several distinct functional roles: a single PLC rack, for example, may be the ControlUnit for three unrelated loops at once (`PressureLoop-3.CU`, `TempLoop-7.CU`, `MeshNode-B.CU` in a Mesh Assembly), each with its own `id`, `tag`, and `state`, all three pointing back to the same `physicalRef`. Conceptually this mirrors the Proxy/Flyweight pattern in OOP: several instances, each with their own identity and interface, sharing one underlying implementation they don't own. It also gives Sub-model 3's **Integrated** topology a checkable definition rather than an assumed one: when every role in one Assembly's chain (S→T→CU→SC→A) shares the same `physicalRef`, that *is* what Integrated means.

`physicalRef` targets whichever granularity is actually the serviceable unit — not always the whole Device. A PLC rack with a single CPU is one shared asset, so `physicalRef` there points to the rack `Device` itself. A rack with **two independent CPUs** is not one shared asset — patching CPU-A never affects loops running on CPU-B — so `physicalRef` should point to the specific `Processor` `Component` (already a standard Component example, Sub-model 1 §5.2) rather than the rack. If `PressureLoop-3.CU` and `TempLoop-7.CU` both run on CPU-A while `MeshNode-B.CU` runs on CPU-B, the first two share one `physicalRef` and the third has a different one, even though all three physically live in the same rack. This works without any new rule: `Component` already carries its own `id` and `tag` (inherited from `LunexObject`) — "not independently addressable" (Sub-model 1 §5.2) describes network reachability, not identity, so a `Component` is just as valid a `physicalRef` target as a `Device`.

`Component` (abstract, extends `LunexObject`) — a part of a Device that is *not* independently addressable (e.g., a digital input channel on a PLC card). Examples: Digital Input, Analog Output, PID-controller, Processor, Comms Module / Gateway.

Five universal device classes (all extend `Device`), distinguished by composable function-interfaces rather than fixed categories:

Class	Function-interface	Role
`Sensor`	Sensing	physical quantity → raw signal
`Transducer`	Converting	raw signal → usable signal
`ControlUnit`	Controlling	logic / decision
`SignalConverter`	Converting	control signal → physical/power signal
`Actuator`	Actuating	signal → physical action

Function-interfaces are composable, not exclusive — a class may implement more than one. This is why an IoT-capable smart device is not a sixth class: it is an existing class (e.g. `Sensor`) plus a `Comms Module` `Component`, or a fully-Integrated topology (Sub-model 3).

Thirteen peers of `Device` / `Component`, added by later sub-models, each extending `LunexObject` directly (not `Device`):

Peer	Sub-model	Summary
`RecipeExecution`	5	One execution of a Recipe, reusing Sub-model 4's state machine directly
`Interlock`	6	Forces/blocks a Sub-model 4 transition
`Zone`	7	Security grouping with target/achieved SL
`Conduit`	7	Governed crossing between two Zones
`Identity`	8	A human or non-human actor capable of acting on the model
`SecurityIncident`	10	A cybersecurity event, peer of Alarm but its own lifecycle
`ChangeRequest`	13	Governed approval workflow for a change to any object
`ChangePolicy`	13	A standing approval rule a `ChangeRequest` resolves against
`Alarm`	15	Priority = severity × actionability
`GuidanceRecommendation`	17	A set of simulated scenarios
`PredictedEvent`	20	A forward-looking claim, raised into Alarm
`ImprovementRecommendation`	20	Always requires human approval
`Operator`	1	A human referenced by `OperatorRef` elsewhere in the model

`Operator` closes a gap the model had without formalizing: `OperatorRef` is used elsewhere (`AIControlUnit.disabledBy`, Sub-model 21) as if it resolved to something, but nothing ever defined what. It is deliberately minimal:

```
Operator extends LunexObject {
  jobTitle : string
      (e.g. "Control Room Operator", "Safety Engineer", "Maintenance Engineer")
}
```

`jobTitle` is a plain descriptive string, not a permissions or access-control system, and deliberately not named `role`, since `Role` is a class in Sub-model 8 — LUNEX records *who* took an action (via `id`, inherited from `LunexObject`) and *in what capacity* (`jobTitle`), not *what they were authorized to do*. A full role-based access model is a genuinely separate concern from object structure and is intentionally left for a future sub-model, not bolted on here to close this gap. Every other place in this specification that refers informally to "the operator" (acknowledging an alarm, approving an `ImprovementRecommendation`, applying a `GuidanceRecommendation`) is not retroactively required to hold a formal `OperatorRef` — that reference is only mandatory where a schema explicitly types a field as `OperatorRef`, as `AIControlUnit.disabledBy` already does.

Color/marking convention (used consistently across all sub-model diagrams): a derived class marked red-and-bold is safety-related (SIL/Interlock); marked blue-and-bold is security-related (IEC 62443).

5.3 Application Guidance

To model a new physical device:

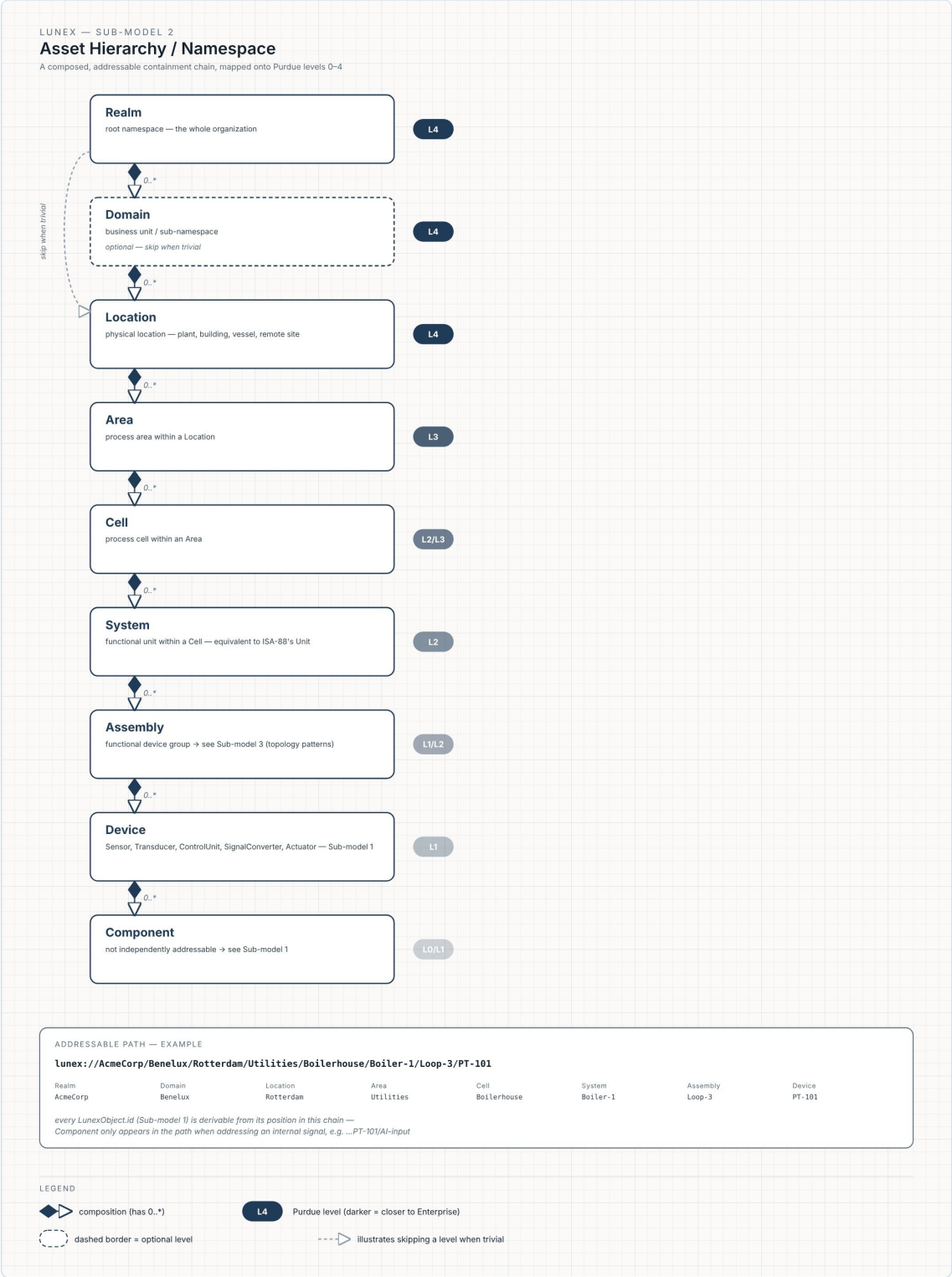
1. Identify its function(s) — Sensing, Converting, Controlling, Actuating — and pick the matching universal class(es). Most real devices need only one; an intelligent transmitter that also converts is the exception, not the rule. A variable-frequency or servo drive is a canonical multi-interface case: it is always both a `ControlUnit` (it computes the output from setpoint and feedback) and a `SignalConverter` (it produces the variable-frequency power signal that drives the motor), and a `Transducer` too if its processed feedback is exposed externally rather than staying internal to its own loop.
2. Decide whether it is independently addressable on the network (`Device`) or a sub-part of one (`Component`).
3. Define it as a derived class in your organization's Inventory (a reusable template — see Sub-model 12's reasoning for why "Inventory" rather than "Library"), not as a one-off instance.
4. Only introduce a new peer class (alongside Interlock, Zone, etc.) if the object genuinely needs its own identity, state, and audit trail independent of any Device — most new concerns should first be checked against whether they can be a

`Component`, a `property`, or an `Assembly` capability flag instead.

5. Set `physicalRef` only when it's actually needed — when one physical asset genuinely backs more than one functional tag. Don't set it by default; an unnecessary `physicalRef` implies a sharing relationship that doesn't exist and will mislead maintenance planning rather than help it.

6. Sub-model 2 — Asset Hierarchy / Namespace

Diagram: `lunex-asset-hierarchy.svg`



lunex-asset-hierarchy.svg

6.1 Purpose

An object model is only useful if every instance can be found. ISA-95 provides a hierarchy, but one scoped to enterprise manufacturing; LUNEX needed a namespace broad enough to address anything from a multinational's plant network down to a single I/O channel, in a form suited to machine addressing (not just human reporting).

6.2 Technical Description

A **composed containment chain**, each level composing zero-or-more of the next:

```
Realm → Domain → Location → Area → Cell → System → Assembly → Device → Component
```

Level	Purdue	Role
Realm	L4	root namespace — whole organization
Domain	L4	business unit (optional — skip when trivial)
Location	L4	plant, building, vessel, remote site
Area	L3	process area within Location (ISA-95 term, reused)
Cell	L2/L3	process cell (ISA-88 term, reused)
System	L2	functional unit within a Cell (equivalent to ISA-88's Unit)
Assembly	L1/L2	topology pattern (Sub-model 3)
Device	L1	one of the five universal classes (Sub-model 1)
Component	L0/L1	not independently addressable

Any level may be skipped when trivial (Domain is the most commonly skipped). Non-production concerns (utilities, safety, IoT) use the same `Area` level as production — there is no separate branch. Every level in this chain is addressable the same way a `Device` is, so `LunexObject.coordinates` (Sub-model 1 §5.2) applies here too — a `Location` can carry the site's own coordinates without needing a separate field, and any object beneath it (a remote valve, a distributed turbine) can independently set its own when the site-level coordinate isn't precise enough.

Addressable path: every `LunexObject.id` is derivable from its position in this chain:

```
lunex://AcmeCorp/Benelux/Rotterdam/Utilities/Boilerhouse/Boiler-1/Loop-3/PT-101
```

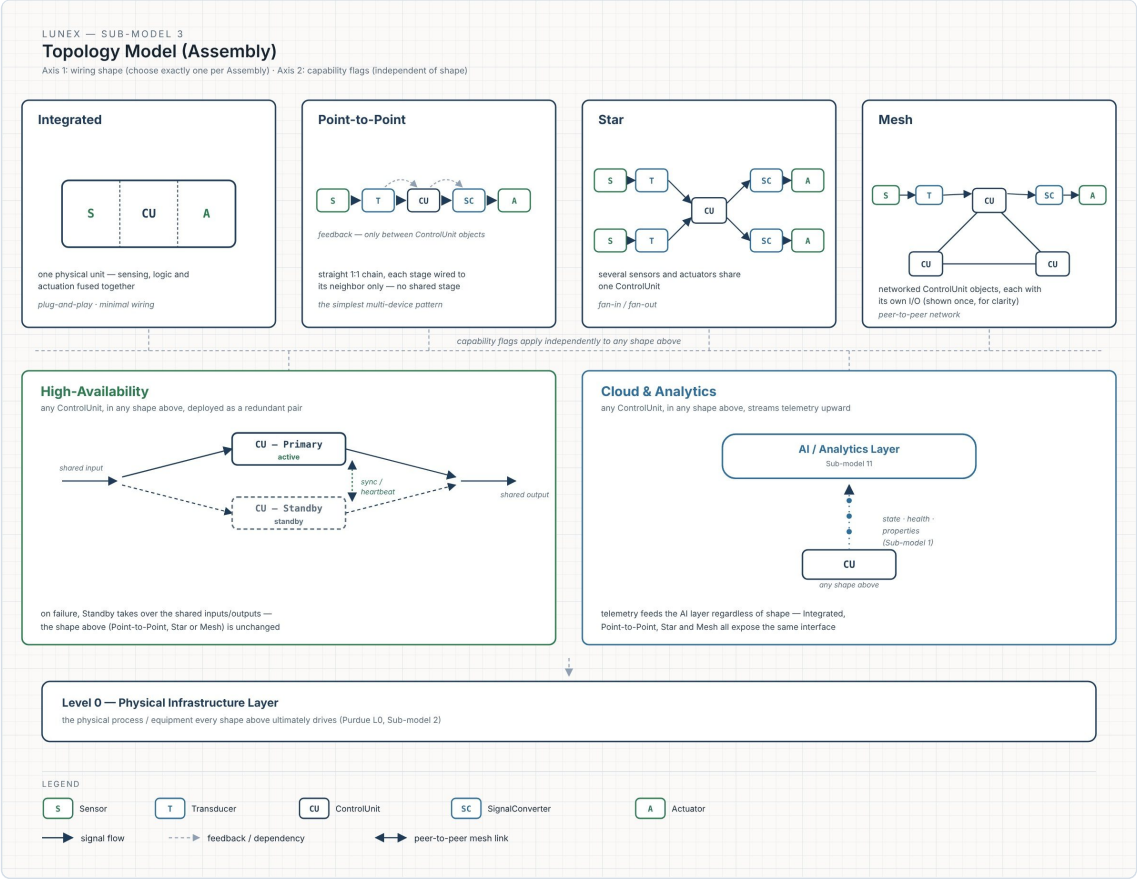
`Component` appears in the path only when addressing an internal signal.

6.3 Application Guidance

1. Start from the top (Realm) and work down; skip Domain immediately if the organization has no meaningful business-unit split relevant to OT addressing.
2. Assign one `Area` per physically or organizationally distinct process zone — utilities, safety systems, and IoT devices each get their own Area rather than a special-cased branch.
3. Use the resulting path directly as the `path` field in the Sub-model 11 Telemetry Envelope — no separate addressing scheme is needed.
4. When integrating with an existing ISA-95 deployment, map `Area` and `Cell` directly (they are the same concept); map `Enterprise` / `Site` to `Realm` / `Location`, noting the broadened scope.

7. Sub-model 3 — Topology Model (Assembly)

Diagram: `lunex-topology-model.svg`



lunex-topology-model.svg

7.1 Purpose

PackML and most DCS documentation implicitly assume a single wiring pattern (roughly Point-to-Point). Real plants mix integrated smart devices, star-wired I/O, and networked mesh controllers within the same facility, often the same Cell. Sub-model 3 makes wiring shape an explicit, independent property of an Assembly rather than an unstated assumption.

7.2 Technical Description

Axis 1 — Wiring shape (exactly one per Assembly):

Shape	Pattern
Integrated	Sensor + ControlUnit + Actuator as one physical unit
Point-to-Point	1:1 linear chain: Sensor → Transducer → ControlUnit → SignalConverter → Actuator
Star	Multiple sensors/actuators share one ControlUnit (fan-in/fan-out)
Mesh	Multiple ControlUnit objects networked peer-to-peer, each with its own I/O

All four shapes include the full S→T→CU→SC→A chain — Transducers and SignalConverter objects are present in Star and Mesh too, not just Point-to-Point.

Axis 2 — Capability flags (0, 1, or both; independent of shape):

- **High-Availability (HA)** — ControlUnit(s) deployed as an active/standby redundant pair.
- **Cloud & Analytics** — ControlUnit(s) stream telemetry to the AI/Analytics layer (Sub-model 11).

An `AIControlUnit` (Sub-model 21) occupies a ControlUnit slot in any of these shapes — most commonly Point-to-Point or Star — without requiring a new shape.

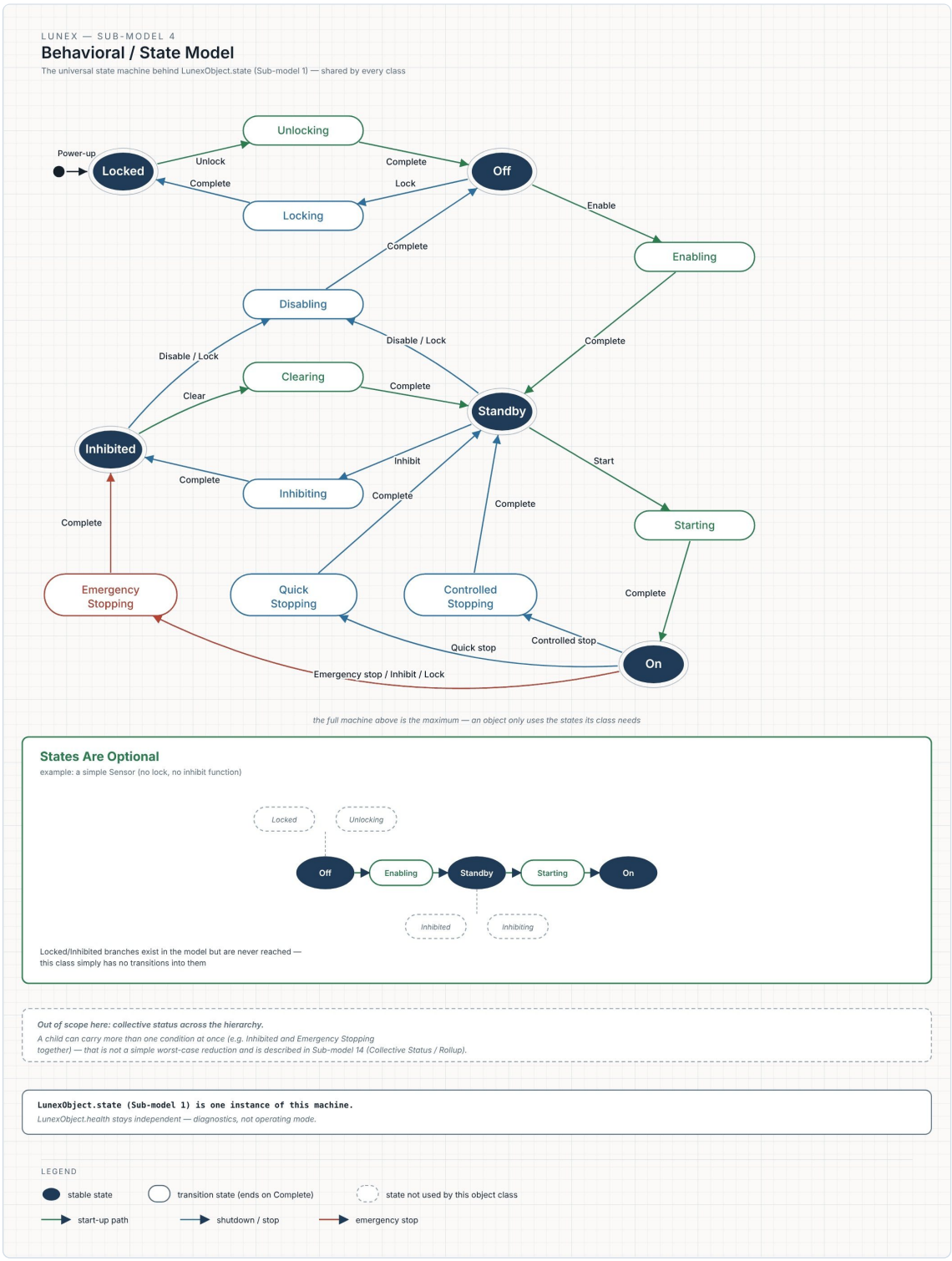
Integrated, made checkable: "one physical unit" no longer has to be an assumption. When every role present in an Assembly's chain shares the same `Device.physicalRef` (Sub-model 1), that shared reference *is* the Integrated shape — a Mesh Assembly whose three nodes each happen to be their own separate hardware is not Integrated no matter how tightly they're networked; a Point-to-Point chain whose S/CU/A all resolve to one `physicalRef` is Integrated even if no one flagged it as such.

7.3 Application Guidance

1. For each Assembly, first pick the wiring shape by asking: does one ControlUnit own this signal chain alone (Point-to-Point/Star), do multiple ControlUnit objects coordinate (Mesh), or is it one sealed unit (Integrated)?
2. Then, independently, decide HA and Cloud & Analytics per Assembly — these are operational/business decisions, not wiring decisions, and should not force a shape change.
3. An Assembly flagged `independent: true` (Sub-model 6, SIF) must never share a ControlUnit with a non-independent Assembly, regardless of shape.

8. Sub-model 4 — Behavioral / State Model

Diagram: `lunex-state-model.svg`



lunex-state-model.svg

8.1 Purpose

PackML defines a state machine for packaging machines. LUNEX needs the equivalent for *any* object class — a Sensor, a ControlUnit, an Interlock, an AIControlUnit — sharing the same vocabulary so that “what state is this in” always means the same thing, regardless of what kind of object is being asked.

8.2 Technical Description

A single state machine, shared by every `LunexObject`, backing `LunexObject.state`:

Stable states: Locked, Off, Standby, Inhibited, On **Transition states**: Unlocking, Locking, Enabling, Disabling, Starting, Inhibiting, Clearing, Quick Stopping, Controlled Stopping, Emergency Stopping

Key transition paths: - Start-up: Locked → Unlocking → Off → Enabling → Standby → Starting → On - Shutdown: On → (Controlled/Quick/Emergency Stopping) → Standby or Inhibited - Lock/Unlock: Locked ↔ (Unlocking/Locking) ↔ Off - Inhibit/Clear: Standby ↔ (Inhibiting/Clearing) ↔ Inhibited - Disable: Standby or Inhibited → Disabling → Off

Transition labels reflect every purpose that triggers them, since Sub-model 6's Interlock reuses these exact transitions rather than inventing new ones: - `Emergency stop / Inhibit / Lock` — the On → Emergency Stopping transition, used by a genuine emergency stop, an inhibit-type Interlock, and (as the first leg) a lock-type Interlock, all starting from On. - `Disable / Lock` — the Standby/Inhibited → Disabling transition, used by a genuine disable action and by a lock-type Interlock's path to Off → Locked.

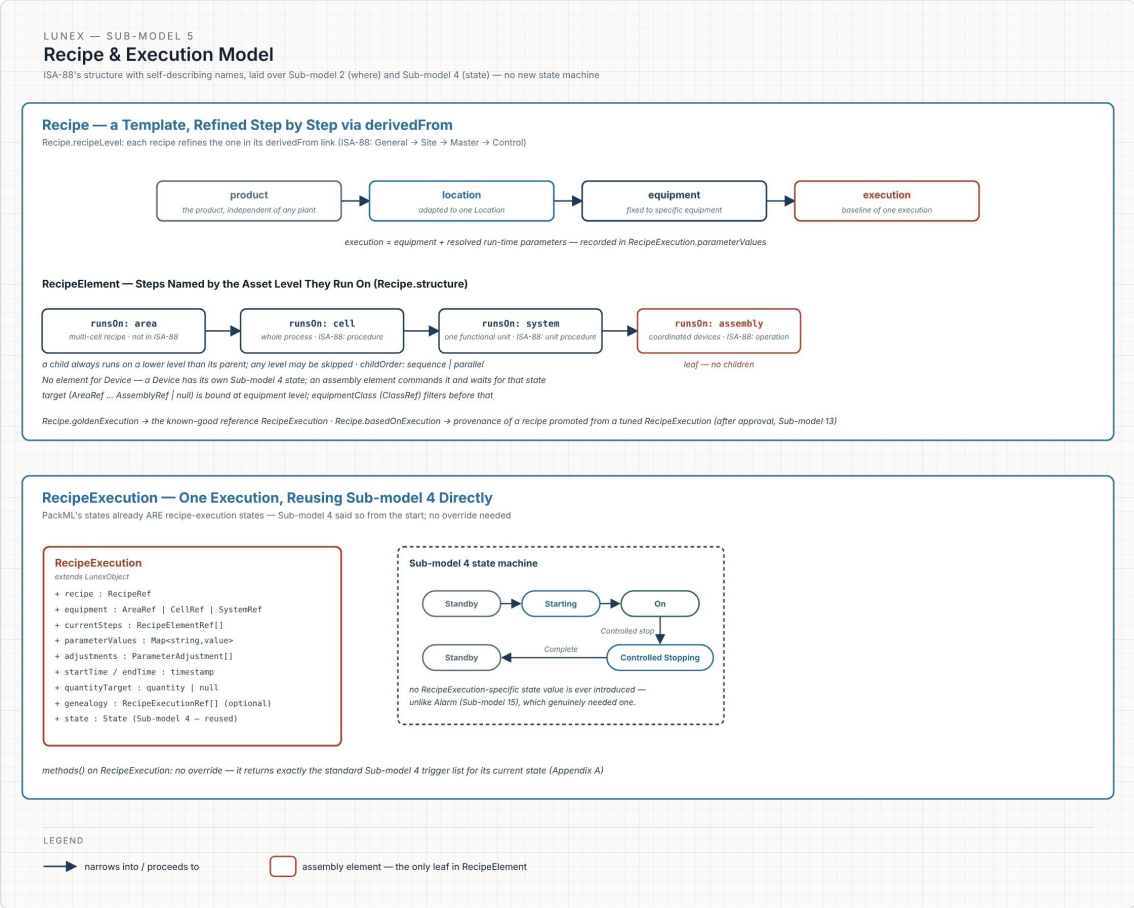
Two design decisions made explicit: 1. **States are optional per object capability.** A simple Sensor only ever uses Off → Enabling → Standby → Starting → On; the Locked/Inhibited branches exist in the model but are simply never reached for that class. 2. **State is always independently owned.** A parent's own state is never overridden by a child's state. Collective status across a hierarchy is a separate, deliberately deferred concern — resolved in Sub-model 14, not here.

8.3 Application Guidance

1. When defining a new derived class, identify only the states it actually needs — do not force every class through the full 14-state machine if, e.g., it has no safety-lockout requirement.
2. Do not build a parallel or simplified state machine for a new object type (as was mistakenly done in an early draft of Sub-model 21) — transplant this state machine and highlight only the relevant paths. This guarantees any correction made here propagates everywhere it's reused.
3. Do not attempt to compute a hierarchy-wide "worst state" from this sub-model alone; see Sub-model 14.

9. Sub-model 5 — Recipe & Execution Model

Diagram: `lunex-recipe-execution-model.svg`



lunex-recipe-execution-model.svg

9.1 Purpose

ISA-88 defines a procedural control model — a chain of recipe levels from product-wide down to one specific run, and a procedural decomposition underneath it, bound to the equipment levels that execute each step — that nothing in LUNEX so far represents. Sub-model 2 answers *where* equipment lives in the plant; Sub-model 4 answers *what state* any one object is in. Neither answers *what sequence of steps a piece of equipment is currently executing, against which recipe, with which parameters* — the question a batch record exists to answer. That is the gap this sub-model fills.

9.2 Technical Description

`Recipe` — a template, not a running instance. LUNEX keeps ISA-88's four recipe levels, renamed so each says what it is, and replaces ISA-88's four fixed step names with elements named by the asset level they run on (the ISA-88 originals are noted once, here, for interoperability):

```
Recipe {
  id          : string
  recipeLevel : product | location | equipment | execution
      (ISA-88: General | Site | Master | Control recipe)
  derivedFrom : RecipeRef | null
      (the less-specific recipe this one refines;
       null only for recipeLevel: product)
  appliesTo   : LocationRef | AreaRef | CellRef | SystemRef | null
      (Sub-model 2 — what this recipe is tied to; must agree with recipeLevel)
  basedOnExecution: RecipeExecutionRef | null
      (set when this recipe was created by promoting a tuned execution — provenance
       only, not a derivedFrom link)
  goldenExecution : RecipeExecutionRef | null
      (the known-good reference execution for this recipe, if one has been
       designated)
  structure   : RecipeElementRef                    (root of the recipe structure)
  parameters  : ParameterSpec[]
      (formula parameters: setpoints, targets, tolerances)
  version     : string
}

RecipeElement {
  id             : string
  runsOn         : area | cell | system | assembly
      (Sub-model 2 — the asset level that executes this step; ISA-88: Procedure |
       Unit Procedure | Operation)
  target         : AreaRef | CellRef | SystemRef | AssemblyRef | null
      (the specific asset; null until the recipe is bound to equipment)
  equipmentClass : ClassRef | null
      (Sub-model 1 Inventory class the target must be an instance of — the filter
       that works before a target is chosen)
  children       : RecipeElementRef[]
      (each runs on a lower level than this element; empty for an assembly element)
  childOrder     : sequence | parallel
      (how the children execute relative to each other)
}
```

How `recipeLevel` and `derivedFrom` work together. `derivedFrom` is the link: each recipe refines exactly one less-specific recipe. `recipeLevel` is the label that makes the chain filterable — "show me every `equipment`-level recipe" — and it must agree with `appliesTo`:

`recipeLevel`	What it is	`appliesTo`	`derivedFrom`
`product`	the product and process, independent of any plant	`null`	`null`
`location`	adapted to the constraints of one Location (Sub-model 2)	`LocationRef`	a `product` recipe
`equipment`	fixed to specific equipment	`AreaRef`, `CellRef` or `SystemRef`	a `location` recipe
`execution`	the recipe as started: the baseline one `RecipeExecution` executes (structure fixed; parameters may be adjusted, each adjustment logged)	the executing `AreaRef` / `CellRef` / `SystemRef`	an `equipment` recipe

A `RecipeExecution` always points at a `execution`-level recipe; the chain back through `derivedFrom` is its full provenance, and its resolved parameter values are recorded in `RecipeExecution.parameterValues`. Product *variants* (grade, size, customer specification) are not a level: they are sibling recipes branching from the same `derivedFrom` parent, at whichever level applies.

Why these names. The ISA-88 originals (General/Site/Master/Control recipe; Procedure/Unit Procedure/Operation/Phase) each collided with something already in LUNEX or with object-oriented vocabulary: `site` is a hierarchy level (Sub-model 2), `control` is a ControlUnit (Sub-model 1), `unit` means `System`, `procedure` is AlarmResponseProcedure (Sub-model 16), `operation` / `method` are object-oriented terms (cf. `methods()`), `scope` already means the target of a

`Permission` / `AccessGrant`, and `generic` is a programming term. The recipe levels therefore name what the recipe is tied to (`location` deliberately reuses the Sub-model 2 level it applies to), and the steps need no vocabulary of their own at all: a step is an `area`, `cell`, `system` or `assembly` element, using the Sub-model 2 names. See Sub-model 12.

How the recipe structure binds to the asset hierarchy. A `RecipeElement` is a step performed by one piece of equipment, and `runsOn` says which kind. The rule is simple: *a child always runs on a lower level than its parent* (area → cell → system → assembly), and any level may be skipped when trivial, exactly as in Sub-model 2. An `area` element is for a recipe spanning several Cells — for example a process cell followed by a packaging line — which ISA-88 does not model; the same rule would admit a `location` level later without redesign. A `cell` element corresponds to ISA-88's procedure, a `system` element to its unit procedure (a `System` is the S88 Unit), and an `assembly` element to its operation. There is deliberately **no element for a `Device`**: a Device carries its own Sub-model 4 state, so an `assembly` element's job is to command its Devices (setpoints, start, stop) and wait for their states, not to model them a second time. `childOrder` makes sequencing explicit: `sequence` runs the children one after another, `parallel` runs them together.

`RecipeExecution` (extends `LunexObject`; peer of Device, Interlock, Zone, Conduit, Alarm, GuidanceRecommendation, PredictedEvent, ImprovementRecommendation, Operator) — the instance, one per execution:

```
RecipeExecution extends LunexObject {
  recipe           : RecipeRef
  equipment        : AreaRef | CellRef | SystemRef
      (Sub-model 2 — the equipment running the root of the recipe structure)
  currentSteps     : RecipeElementRef[]
  parameterValues  : Map<string, value>
      (resolved formula values at start of execution — the genealogy record)
  adjustments      : ParameterAdjustment[]
      (every change made while running, in order — empty if none)
  startTime        : timestamp
  endTime          : timestamp | null
  quantityTarget   : quantity | null
      (planned amount, UCUM unit; null = continuous or open-ended production)
  genealogy        : RecipeExecutionRef[]
      (prior executions whose output material fed this one, when applicable —
       optional)
}
```

```
ParameterAdjustment {
  parameter  : string
  previous   : value
  new        : value
  at         : timestamp
  by         : IdentityRef | AIControlUnitRef
      (Sub-model 8 / Sub-model 21 — a human or the AI that tuned it)
  basis      : string | null
      (e.g. a Digital Twin simulation reference, Sub-model 11 / 17)
}
```

Tuning during a run. A `RecipeExecution` may be adjusted while it executes — by an operator, or by an AI using the Digital Twin — and two cases must be kept apart. (1) *Adjustment within the recipe's approved bounds* is normal operation, not a change: it is appended to `RecipeExecution.adjustments`, and an `AIControlUnit` may only act inside its `operatingBounds` (Sub-model 21). (2) *Promoting a tuned result to a new recipe* is a change: it is raised as an `ImprovementRecommendation` (Sub-model 20, always human-approved), goes through a `ChangeRequest` (Sub-model 13), and produces a new `Recipe` whose `derivedFrom` is the original's parent and whose `basedOnExecution` names the execution it came from. An execution whose result is the best-known execution can be designated `Recipe.goldenExecution`; "golden" carries the same meaning as the golden scenario in Sub-model 17 — a known-good reference to compare against.

`RecipeExecution.state` is deliberately not a new value space. Sub-model 4 is already, by its own purpose statement, "a generalization of PackML's state pattern to any object class" — and PackML's states (Idle/Starting/Running/Holding/Held/Aborting/Complete, in Sub-model 4's vocabulary: Standby/Starting/On/Inhibiting/Inhibited/Emergency Stopping/On-returning-to-Standby) are literally batch-execution states. Unlike `Alarm` (Sub-model 15), which needed its own state space because an operator-response workflow answers a different question than object behavior, a recipe execution *is* object behavior — so it reuses Sub-model 4's machine directly, with no override of `methods()`. This is the sub-model's only structural claim: not a new state model, only a procedural/recipe structure and a genealogy record laid over the state model that already exists.

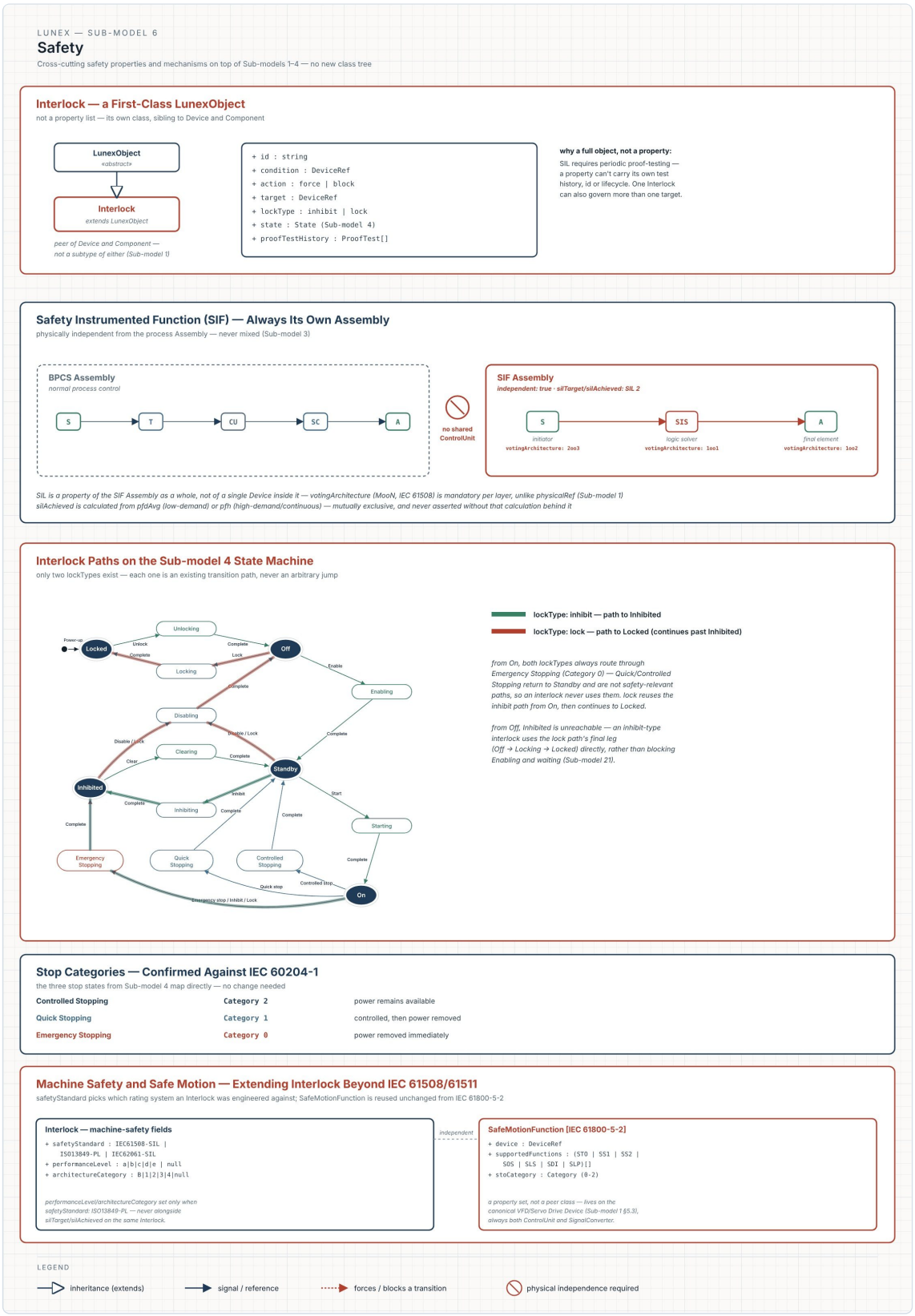
A `RecipeExecution`'s `currentSteps` tracks position in the static `RecipeElement` tree (several steps at once when a parent's `childOrder` is `parallel`); the equipment actually executing a step (its `target`, Sub-model 2) carries its own Sub-model 4 state exactly as it always has — a `RecipeExecution` does not duplicate or shadow that state, it only points at where in the recipe the equipment currently is.

9.3 Application Guidance

1. Author one `Recipe` per product/process, at `recipeLevel: equipment`, with parameter values left unresolved; resolve them into a `RecipeExecution.parameterValues` only at execution time (the `execution`-level recipe) — do not pre-resolve parameters into the `equipment` recipe itself.
2. Do not create a runtime object per `RecipeElement` — a step's execution state is its `target`'s own Sub-model 4 state; tracking it twice invites the two from drifting apart. Check on bind that `target` is an instance of `equipmentClass`, and that every child's `runsOn` is lower than its parent's.
3. Populate `genealogy` only where traceability is actually required (e.g., regulated or safety-critical product) — it is optional for a reason, not a mandatory audit trail for every execution.
4. A change to an `equipment`- or `execution`-level Recipe affecting a live or future `RecipeExecution` should be routed through a `ChangeRequest` (Sub-model 13 — Management of Change), not edited in place.
5. Use `RecipeExecution` for any production form, not only batches: a discrete lot has a `quantityTarget`; a campaign is one execution spanning several lots; a continuous process is one execution with `quantityTarget: null` and `endTime: null`, whose steady state is a `system` element with no end condition. A changeover is a new execution on a derived recipe, not an edit.
6. Record every in-run change in `RecipeExecution.adjustments` — including AI-made ones — and never overwrite `parameterValues`; only a reviewed, approved promotion creates a new `Recipe` (with `basedOnExecution`), never an in-place edit of the recipe the execution ran.

10. Sub-model 6 — Safety

Diagram: `lunex-safety-model.svg`



lunex-safety-model.svg

10.1 Purpose

Functional safety (IEC 61508/61511) needs traceable, auditable interlock logic — proof-test history, independence from the BPCS, and precise, unambiguous behavior. Modeling a safety interlock as a mere property on a Device cannot carry that history or independence; it needs to be a first-class object in its own right. This version also closes two gaps found when the model was checked against **machine** safety specifically, as distinct from the **process** safety IEC 61508/61511 already covers: a SIF's integrity claim had no achieved/verified counterpart, and nothing represented ISO 13849-1/IEC 62061 (the parallel rating system used by discrete/machine safety) or IEC 61800-5-2's drive-level safe-motion functions.

10.2 Technical Description

`Interlock` (extends `LunexObject` directly — a peer of `Device`, not a subtype of it):

```
Interlock {
  id                : string
  condition         : DeviceRef
  action            : force | block
  target            : DeviceRef
  lockType          : inhibit | lock
  state             : State               (Sub-model 4)
  proofTestHistory  : ProofTest[]
}
```

`Interlock` overrides `methods()` (Sub-model 1 §5.2.1) rather than inheriting the standard Sub-model 4 trigger list: `clear()` — releases a tripped condition once the underlying cause is gone. This is the Interlock's own `clear()`, distinct from the *target* Device's own `clear()` trigger (which moves the target from Inhibited back to Standby once the Interlock permits it).

`lockType` and its target, by the object's current state (this is the corrected, complete rule — see §10.4):

Current state	`lockType: inhibit` targets	`lockType: lock` targets
On	Inhibited (via Emergency Stopping)	Locked (via Emergency Stopping → Inhibited → Disabling → Off → Locking)
Standby	Inhibited (via Inhibiting)	Locked (via Disabling → Off → Locking)
Inhibited	*(already there)*	Locked (via Disabling → Off → Locking)
Off	Locked (via Locking) — Inhibited is unreachable from Off	Locked (via Locking)

Both `lockType` values, when the current state is Off, converge on the same result: **Locked**. This is not two competing behaviors — it follows directly from the Sub-model 4 transition graph having no Off→Inhibited edge.

Safety Instrumented Function (SIF): modeled as an Assembly (Sub-model 3) with `independent: true`. Topology is Point-to-Point: Sensor (initiator) → ControlUnit (logic solver, typically an SIS) → Actuator (final element). A SIF Assembly must never share a ControlUnit with a BPCS Assembly.

`silTarget` / `silAchieved` — the SIF integrity claim, split the same way Sub-model 7's `securityLevelTarget` / `securityLevelAchieved` already splits a Security Level claim. An earlier version of this sub-model carried a single `SIL` field — a gap found on review: every other integrity claim in LUNEX (Security Level here, the Tier-based `riskAssessment` on a `ChangeRequest`, Sub-model 13) distinguishes what was targeted from what was actually achieved, and SIL did not.

```
SIF Assembly {
  silTarget      : SIL (1-4)
  silAchieved    : SIL (1-4)
      (derived from pfdAvg/pfh against the IEC 61508 SIL band table)
  pfdAvg         : number | null    (low-demand mode)
  pfh            : number | null
      (high-demand / continuous mode — mutually exclusive with pfdAvg)
  votingArchitecture : {
    sensor         : MooN     (e.g. "2oo3")
    logicSolver    : MooN     (e.g. "1oo1" or "1oo2")
    finalElement   : MooN     (e.g. "1oo2")
  }
}
```

`MooN` is IEC 61508's own voting notation, reused unchanged. Distinct from Sub-model 3's `High-Availability` flag: HA is about uptime and applies to any Assembly, BPCS or SIF; `votingArchitecture` is about SIL integrity — it feeds the `pfdAvg` / `pfh` calculation and applies only to a SIF. The two are independent and can both be present on the same Assembly for different reasons.

`votingArchitecture` is **mandatory**, not optional like `physicalRef` (Sub-model 1) — the two fields look similar but follow opposite reasoning. Omitting `physicalRef` has a safe, harmless default (the instance is assumed to be its own unique asset). Omitting `votingArchitecture` has no safe default: an implicit `1oo1` would silently assert a SIL-relevant fact — no voting redundancy — that must never be assumed, only explicitly engineered and documented. This is the same principle already governing `AIControlUnit.operatingBounds` (Sub-model 21): mandatory wherever omission would silently carry a real-world consequence, optional only where omission is genuinely inert. `silTarget` is mandatory for the same reason; `silAchieved` / `pfdAvg` / `pfh` may be `null` only before the PFD/PFH calculation has actually been performed — a SIF commissioned without one is a defect to flag (§10.4), not a valid permanent state.

Stop categories [IEC 60204-1], confirmed against Sub-model 4:

LUNEX state	Category
Controlled Stopping	2 — power remains available
Quick Stopping	1 — controlled, then power removed
Emergency Stopping	0 — immediate power removal

Quick and Controlled Stopping both return to Standby and are never used by an Interlock — only Emergency Stopping (Category 0) is a safety-relevant path from On.

Machine safety — ISO 13849-1 / IEC 62061, a second rating system reused alongside IEC 61508/61511, not merged into it: process safety (SIF, SIL) and machine/discrete safety (guarding, E-stops, light curtains) are routinely engineered to different standards on the same site, and ISO 13849-1's informal SIL↔PL correspondence table is indicative, not an equivalence — so this is new fields on `Interlock`, not a relabeling of `silTarget` / `silAchieved`:

```
Interlock {
  safetyStandard     : IEC61508-SIL | ISO13849-PL | IEC62061-SIL
      (which rating system this Interlock was engineered against)
  performanceLevel   : a | b | c | d | e | null
      (ISO 13849-1 — set only when safetyStandard: ISO13849-PL)
  architectureCategory : B | 1 | 2 | 3 | 4 | null
      (ISO 13849-1 architecture category — set only
       when safetyStandard: ISO13849-PL)
}
```

An `Interlock` with `safetyStandard: IEC61508-SIL` leaves `performanceLevel` / `architectureCategory` `null` and is instead backed by the SIF Assembly's `silTarget` / `silAchieved` above — the two rating systems are never populated on the same Interlock at once.

Safe Motion functions [IEC 61800-5-2] — the machine-safety functions that make a VFD/servo drive (Sub-model 1's canonical multi-interface Device example) more than a speed controller, modeled as a property set on the drive rather than a separate `Interlock`, since these are safety functions the drive itself executes internally, not a trip forced by an external condition on an external target:

```
SafeMotionFunction {
  device              : DeviceRef
      (the Drive)
  supportedFunctions  : (STO | SS1 | SS2 | SOS | SLS | SDI | SLP)[]
  stoCategory         : Category (0-2)
      (IEC 60204-1, via the Stop Categories table above — STO is always Category 0)
}
```

`STO` (Safe Torque Off) is the baseline function every safety-rated drive supports and always maps to Stop Category 0; `SS1` / `SS2` (Safe Stop 1/2) are controlled, monitored stops before torque removal, mapping to Category 1; the remaining functions (`SOS` , `SLS` , `SDI` , `SLP`) constrain motion without stopping and have no Stop Category equivalent, since the drive never reaches Off.

10.3 Application Guidance

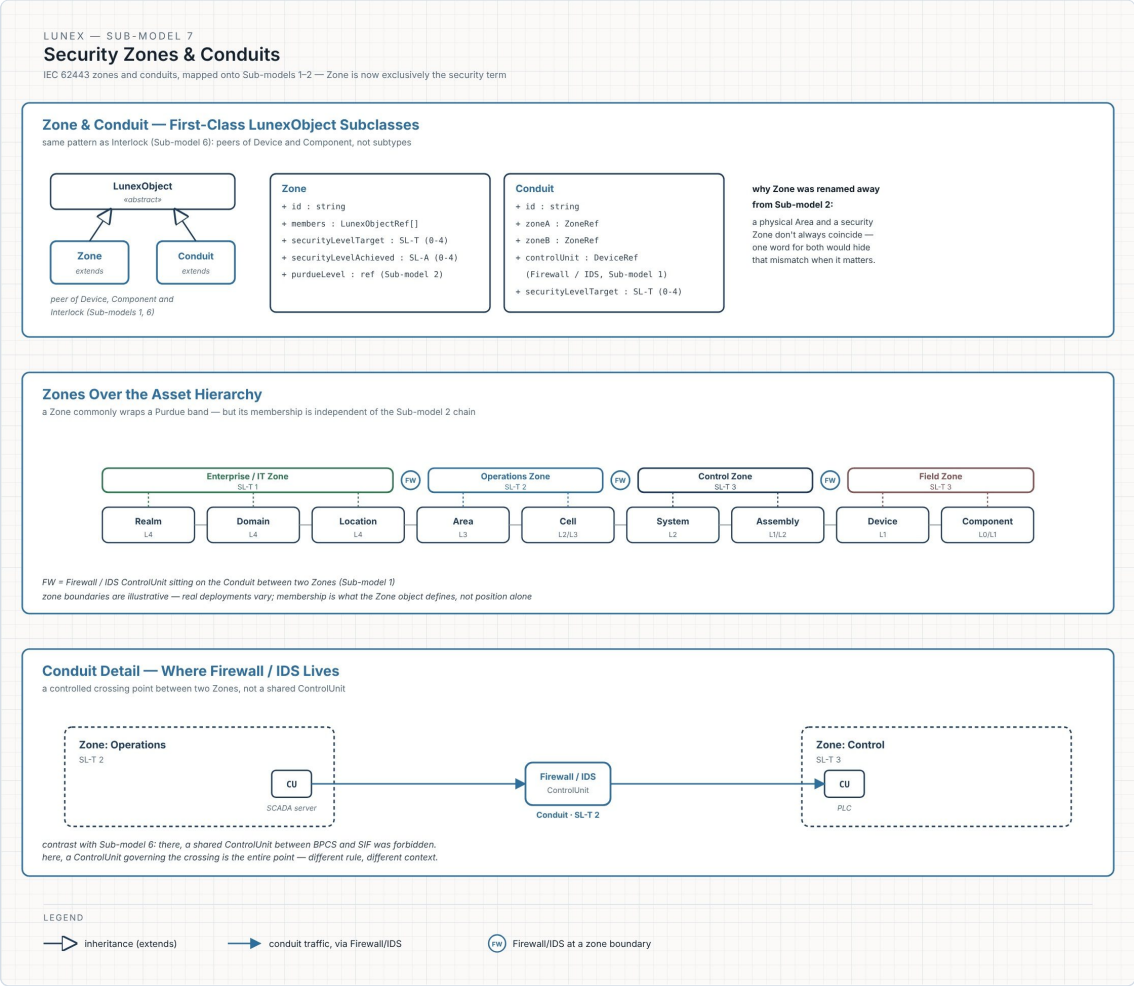
1. For every safety function identified in your IEC 61508/61511 SIL assessment, create one `Interlock` instance per condition→target relationship — do not merge multiple independent trip conditions into a single Interlock, or the proof-test history becomes ambiguous.
2. Choose `lockType: inhibit` when the process should pause but logic can stay live (guard doors, process permissives); choose `lockType: lock` when a full lockout is required before restart (machine guarding, maintenance interlocks).
3. Do not assume the target state from `lockType` alone — always resolve it against the table in §10.2 using the target Device's current state.
4. Any SIF must be assigned to its own Assembly at the point of topology design (Sub-model 3), before any shared-Control-Unit decision is made elsewhere in the project.
5. Set `votingArchitecture` for all three layers explicitly, even when a layer is genuinely `1oo1` — an unset layer is a missing engineering decision, not a valid "no vote needed" default. Voting can differ per layer (e.g. `2oo3` sensors feeding a `1oo1` logic solver and a `1oo2` final element) — this is normal, not an inconsistency to resolve.
6. Treat a SIF with `silTarget` set but `silAchieved` / `pfdAvg` / `pfh` left `null` past commissioning as a configuration defect, the same discipline already applied to a missing ARP on a Priority 1/2 alarm (Sub-model 16) — route it through a `ChangeRequest` (Sub-model 13) before the SIF is relied upon.
7. Pick `safetyStandard` per `Interlock` based on which assessment actually governed it (IEC 61508/61511 SIL assessment, or ISO 13849-1/IEC 62061 machine risk assessment) — do not default every machine-guarding Interlock to `IEC61508-SIL` merely because the rest of the plant uses it.
8. Populate `SafeMotionFunction.supportedFunctions` from the drive's own safety datasheet — do not assume `STO` support implies any of the other functions are also present; they are independently certified capabilities.

10.4 Note on a corrected edge case

An earlier version of this specification stated that `lockType: inhibit` only applied "from Standby," implying it could not be triggered while an object was Off. This was incomplete: on reflection, when an inhibit-type Interlock condition is active while the target is already Off, the correct behavior is not to silently block the Off→Enabling transition and wait — it is to actively drive the object to Locked, using the same Off→Locking→Locked edge that `lockType: lock` already uses. This is now the specified behavior (§10.2) and applies to every object with an Interlock, not only the `AIControlUnit` case (Sub-model 21) where it was first identified.

11. Sub-model 7 — Security Zones & Conduits

Diagram: `lunex-security-model.svg`



lunex-security-model.svg

11.1 Purpose

IEC 62443 requires zone-and-conduit segmentation with defined Security Levels. As with safety, a security zone needs an identity, membership, and a target/achieved SL — properties that don't fit cleanly as attributes on a Device, and that must not be confused with the physical `Area` concept from Sub-model 2.

11.2 Technical Description

`Zone` (extends `LunexObject`):

```
Zone {
  id                    : string
  members               : LunexObjectRef[]
  securityLevelTarget   : SL-T (0-4)
  securityLevelAchieved : SL-A (0-4)
  purdueLevel           : ref                (Sub-model 2)
}
```

`Conduit` (extends `LunexObject`):

```
Conduit {
  id                    : string
  zoneA                 : ZoneRef
  zoneB                 : ZoneRef
  controlUnit           : DeviceRef          (Firewall / IDS, Sub-model 1)
  securityLevelTarget   : SL-T (0-4)
}
```

A Zone's `members` is independent of the Sub-model 2 containment chain — a Zone commonly wraps a Purdue band (e.g., an "Operations Zone" covering Area/Cell) but its actual boundary is defined by membership, not by position in the hierarchy.

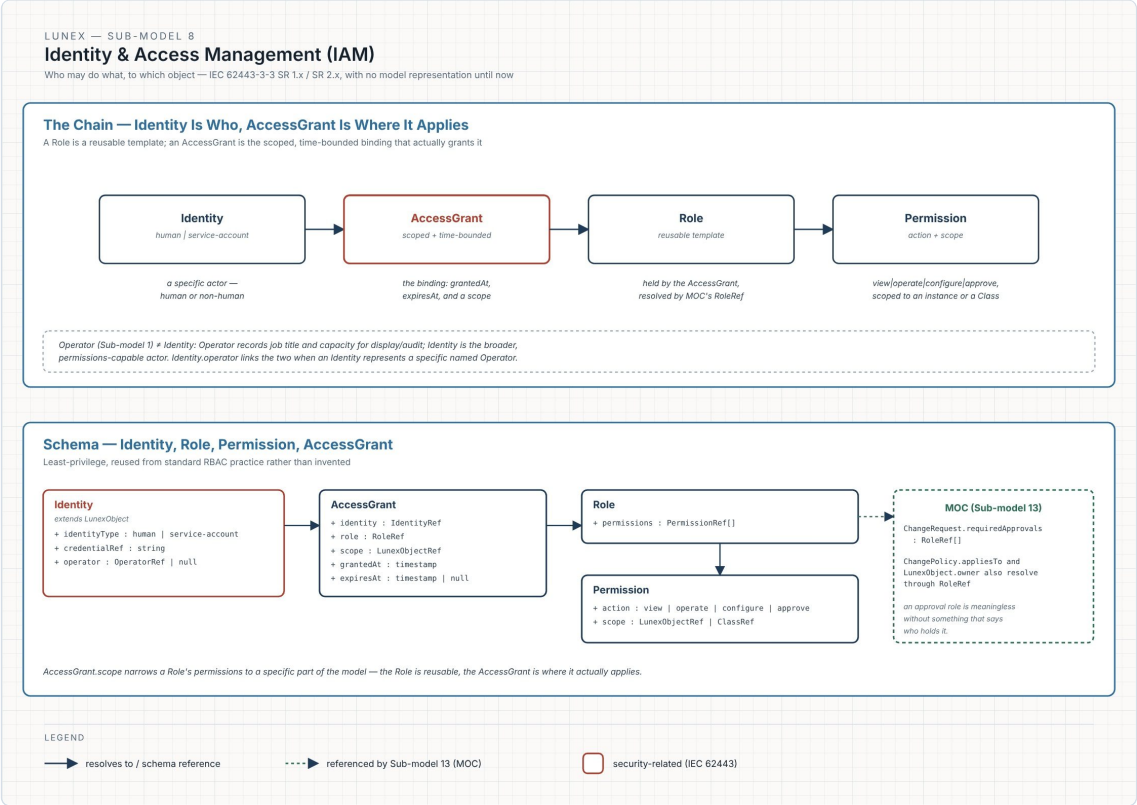
A `Firewall / IDS` (a `ControlUnit` subclass, Sub-model 1, marked blue/security) sits at the Conduit boundary between two Zones — this is a normal, intended use of a shared crossing point, distinct from the Sub-model 6 rule that forbids a *shared ControlUnit between BPCS and SIF Assemblies*. Both rules concern ControlUnit sharing, but in different contexts with opposite intent.

11.3 Application Guidance

1. Perform the IEC 62443 zone-and-conduit exercise first, using standard methodology; then encode its output as `Zone` / `Conduit` instances — LUNEX does not replace that analysis.
2. Do not reuse `Area` (Sub-model 2) as a stand-in for `Zone` even when their boundaries happen to coincide in a given plant — a future re-segmentation of one should not force a re-segmentation of the other.
3. Assign the Firewall/IDS ControlUnit to the Conduit's `controlUnit` field, not to either Zone individually.

12. Sub-model 8 — Identity & Access Management

Diagram: `lunex-iam-model.svg`



lunex-iam-model.svg

12.1 Purpose

IEC 62443-3-3 SR 1.x (identification and authentication) and SR 2.x (use control/authorization) are not represented anywhere in LUNEX. `Operator` (Sub-model 1) records identity and capacity — "who" — but deliberately not permissions; `Zone` / `Conduit` (Sub-model 7) segment the network, not who may act within a segment. Without this sub-model, "who is allowed to do what, to which object" has no answer in the model at all — including the question MOC's `requiredApprovals` (Sub-model 13) needs resolved: an approval role is meaningless without something that says who holds it.

12.2 Technical Description

`Identity` (extends `LunexObject`) — a human or non-human actor capable of acting on the model, distinct from `Operator`'s narrower "job title and capacity" record:

```
Identity extends LunexObject {
  identityType   : human | service-account
  credentialRef  : string
      (pointer to the actual credential store — LUNEX does not model credentials
       themselves)
  operator       : OperatorRef | null
      (Sub-model 1 — when this Identity represents a specific named Operator)
}
```

`Role` and **`Permission`** — least-privilege, reused from standard RBAC practice rather than invented:

```
Role {
  id           : string
  permissions  : PermissionRef[]
}

Permission {
  action  : view | operate | configure | approve
  scope   : LunexObjectRef | ClassRef
      (a specific instance, or an entire Inventory class — Sub-model 1 §5.3)
}
```

`AccessGrant` — the binding between an Identity and a Role, scoped and time-bounded:

```
AccessGrant {
  identity    : IdentityRef
  role        : RoleRef
  scope       : LunexObjectRef
  grantedAt   : timestamp
  expiresAt   : timestamp | null
}
```

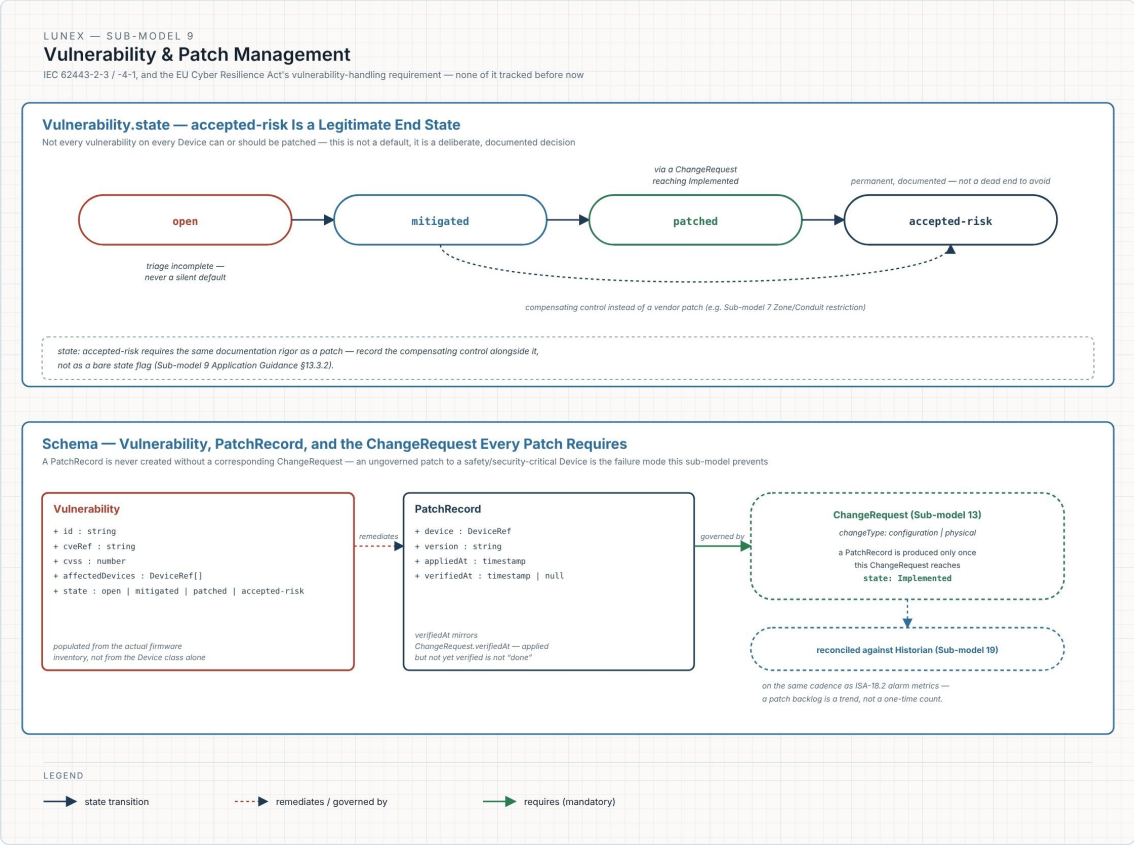
`AccessGrant.scope` narrows a `Role`'s permissions to a specific part of the model (e.g., a Role with `configure` on the `Interlock` class, granted only `scope: Cell-04`) — a Role is a reusable template, an AccessGrant is where it actually applies. MOC's `ChangeRequest.requiredApprovals` (Sub-model 13) is a list of `RoleRef`s pointing back here, rather than a free-text string resolved by convention.

12.3 Application Guidance

1. Model `Role` / `Permission` at the granularity your actual approval and operation workflows need — do not create a one-off Role per Identity; that defeats the purpose of a reusable Role.
2. Set `AccessGrant.expiresAt` for any temporary elevation (contractor access, a vendor support window) — an `AccessGrant` with no expiry for a permanent staff role is normal; one with no expiry for a temporary grant is a finding to flag.
3. Route every `AccessGrant` change for a `configure` / `approve` Permission through a `ChangeRequest` (Sub-model 13) — access escalation is exactly the kind of change MOC exists to govern.
4. Do not store credentials in `credentialRef` — it is a pointer to an external identity provider or credential vault, never the secret itself.

13. Sub-model 9 — Vulnerability & Patch Management

Diagram: `lunex-vulnerability-model.svg`



lunex-vulnerability-model.svg

13.1 Purpose

IEC 62443-2-3 (patch management) and IEC 62443-4-1 (secure product development) both require tracking known vulnerabilities and patch status per asset — and the EU Cyber Resilience Act (applicable from 2027) makes vulnerability handling and an SBOM a legal requirement for connected products. Nothing in LUNEX currently tracks a Device's firmware version, known CVEs, or patch state.

13.2 Technical Description

```
Vulnerability {
  id               : string
  cveRef           : string                (CVE identifier, reused unchanged)
  cvss             : number                (CVSS score, reused unchanged)
  affectedDevices  : DeviceRef[]
  state            : open | mitigated | patched | accepted-risk   (own value space)
}

PatchRecord {
  device       : DeviceRef
  version      : string
  appliedAt    : timestamp
  verifiedAt   : timestamp | null
}
```

`Vulnerability.state: accepted-risk` is a legitimate, permanent end state — not every vulnerability on every Device can or should be patched (a legacy controller with no vendor patch path, where the mitigation is compensating network segmentation via Sub-model 7 instead). It is not a default: an unset status is incomplete triage, not an implicit acceptance.

Every `PatchRecord` is the result of a `ChangeRequest` (Sub-model 13) that has reached `Implemented` — patching a Device is

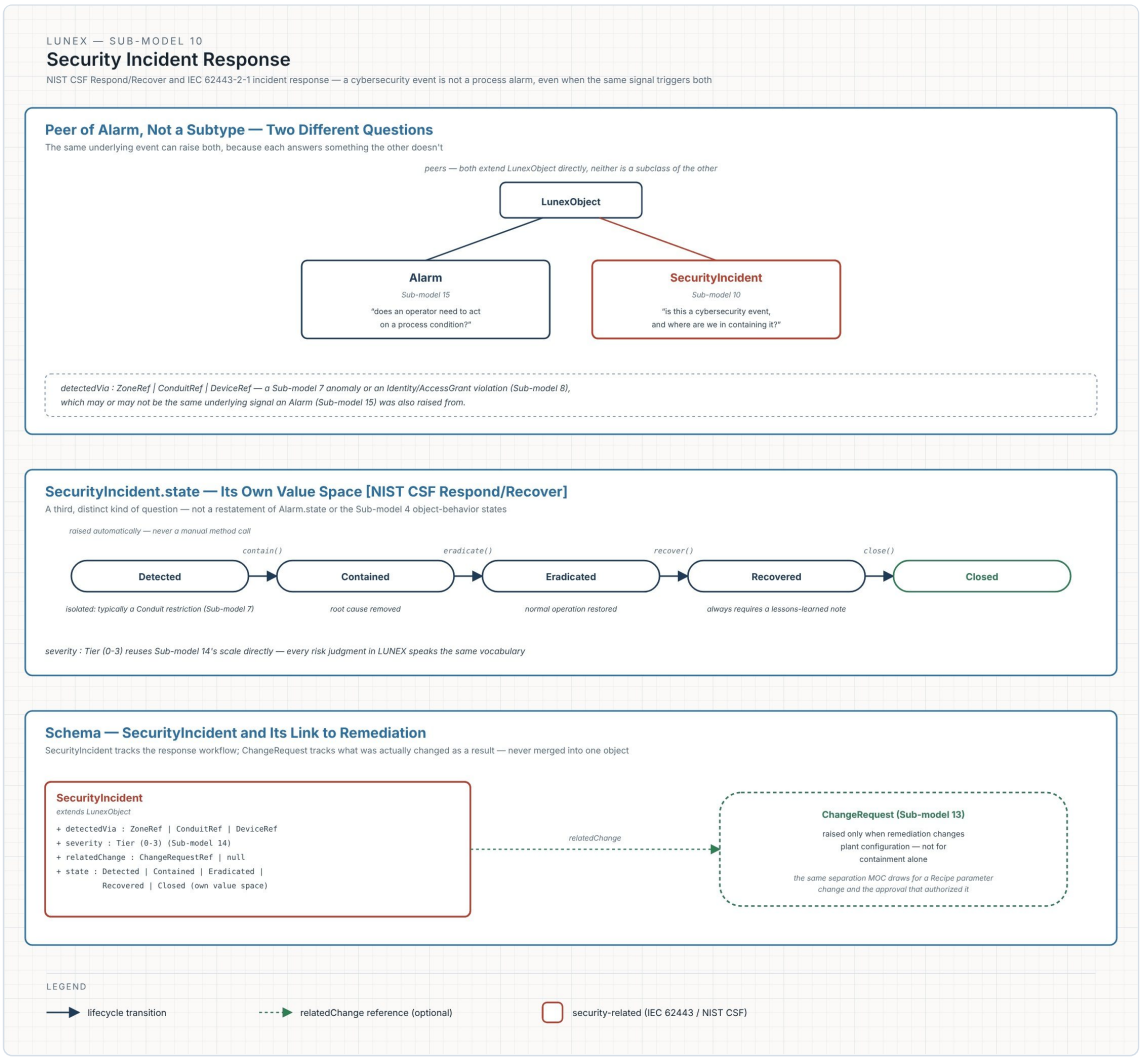
a change like any other, and reuses that governance rather than a parallel patch-approval flow. `verifiedAt` mirrors the `ChangeRequest.verifiedAt` pattern: a patch applied but not yet verified is not the same as one confirmed effective.

13.3 Application Guidance

1. Populate `affectedDevices` from the actual firmware/software inventory, not from the Device class alone — two Devices of the same class can carry different firmware versions and therefore different exposure.
2. Treat `state: accepted-risk` as requiring the same documentation rigor as a patch — record the compensating control (typically a Sub-model 7 `Zone` membership or `Conduit` restriction) alongside it, not as a bare status flag.
3. Reconcile `PatchRecord` against the Historian (Sub-model 19) on the same cadence as the ISA-18.2 alarm metrics — a patch backlog is a trend, not a one-time count.
4. Do not create a `PatchRecord` without a corresponding `ChangeRequest` — an un-governed patch applied directly to a safety- or security-critical Device is the failure mode this sub-model exists to prevent.

14. Sub-model 10 — Security Incident Response

Diagram: `lunex-security-incident-model.svg`



lunex-security-incident-model.svg

14.1 Purpose

Alarm Management (Sub-model 15) is explicitly process-alarm-focused — ISA-18.2's priority/state model answers "does an operator need to act on a process condition," not "is this a cybersecurity event, and where are we in containing it." NIST CSF's Respond/Recover functions and IEC 62443-2-1's incident-response requirements have no object representation anywhere in LUNEX.

14.2 Technical Description

`SecurityIncident` (extends `LunexObject`; peer of `Alarm`, not a subtype of it — a security incident and a process alarm answer different questions even when triggered by the same underlying event):

```
SecurityIncident extends LunexObject {
  detectedVia   : ZoneRef | ConduitRef | DeviceRef
      (Sub-model 7, or the Device itself)
  severity      : Tier (0-3)
      (Sub-model 14 — reused, not a new risk scale)
  relatedChange : ChangeRequestRef | null
      (Sub-model 13 — remediation, once raised)
  state         : SecurityIncidentState                (own value space — see below)
}
```

`SecurityIncident.state` is deliberately its own value space, following the same reasoning that already separates `Alarm.state` from Sub-model 4's object-behavior states: incident-response workflow is a third, distinct kind of question, not a restatement of either.

States: `Detected`, `Contained`, `Eradicated`, `Recovered`, `Closed` [NIST CSF Respond/Recover].

- `detect` → *(none)* → Detected (automatic, from a Sub-model 7 anomaly or an Identity/`AccessGrant` violation, Sub-model 8)
- `contain` → Detected → Contained (the immediate threat is isolated — typically a `Conduit` restriction, Sub-model 7)
- `eradicate` → Contained → Eradicated (root cause removed)
- `recover` → Eradicated → Recovered (normal operation restored)
- `close` → Recovered → Closed (always requires a lessons-learned note — see §14.3)

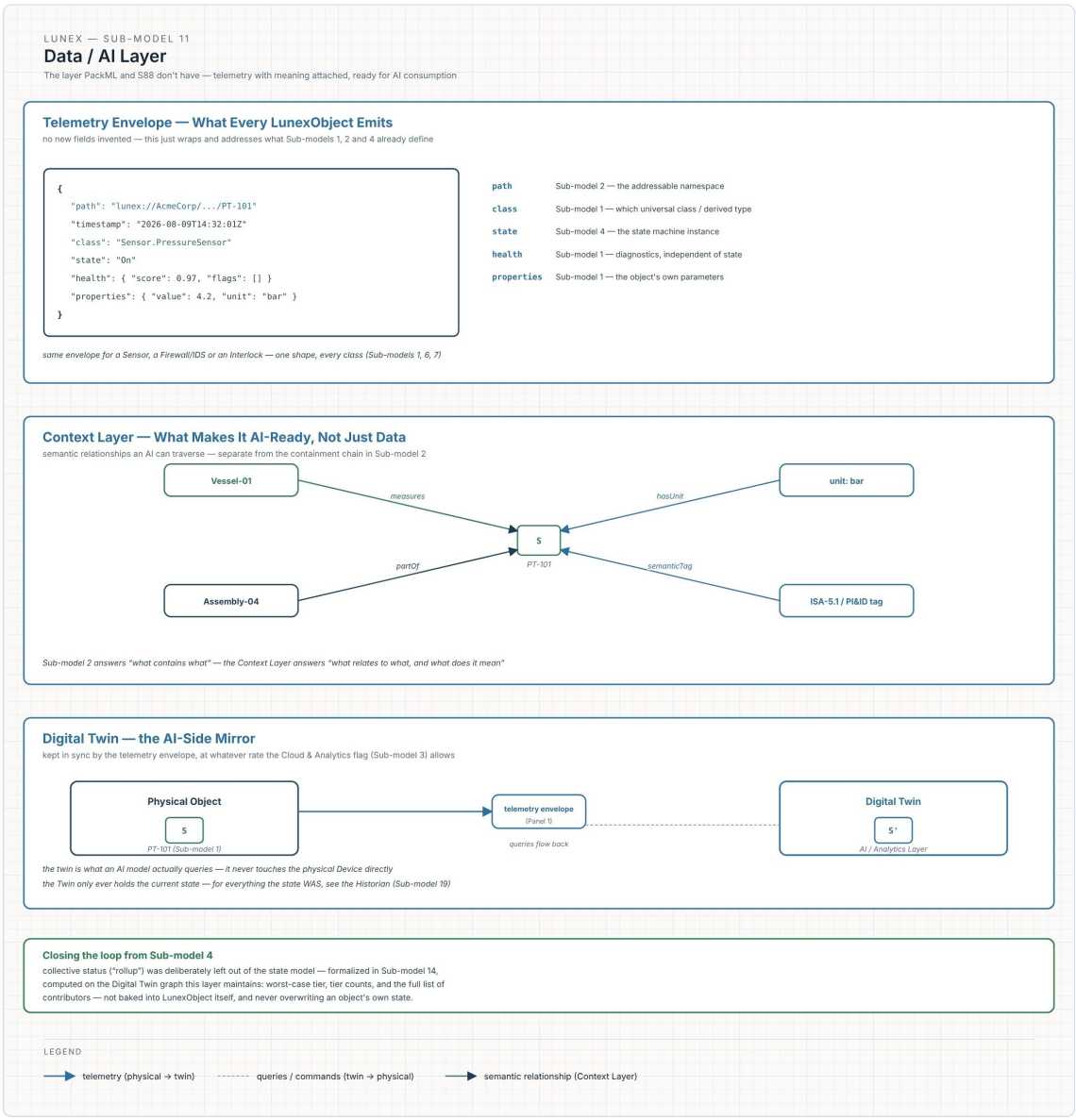
A remediation that changes plant configuration (not just containment) raises a `ChangeRequest` (Sub-model 13), referenced via `relatedChange` — `SecurityIncident` tracks the response workflow, `ChangeRequest` tracks what was actually changed as a result; the two are never merged into one object, the same separation MOC already draws between a Recipe parameter change and the approval that authorized it.

14.3 Application Guidance

1. Raise one `SecurityIncident` per distinct event, even when several are later found to share a root cause — merge the finding into a shared `relatedChange`, not into a single incident record, so each detection retains its own timeline.
2. Set `severity` using the same Tier judgment already applied to Rollup (Sub-model 14) — a Tier 0 `SecurityIncident` gets the same escalation urgency as a Tier 0 process Rollup, not a separate security-specific scale that operators have to learn on top of the one they already use.
3. Never close a `SecurityIncident` without a `relatedChange` when the response required any configuration change — a `Closed` incident with no governed change behind it is a response that was never actually implemented, only contained.
4. Do not reuse `Alarm` for a security event merely because both are first noticed on an operator screen — raise a `SecurityIncident` instead and, if operator awareness is also needed, let Situational Awareness (Sub-model 18) surface it the same way it surfaces a Rollup or a top alarm, without forcing it through the ISA-18.2 Alarm state machine it was never designed for.

15. Sub-model 11 — Data / AI Layer

Diagram: `lunex-data-ai-model.svg`



lunex-data-ai-model.svg

15.1 Purpose

Neither PackML nor ISA-88 anticipated AI as a structural part of the system. Making an object model "AI-ready" is not simply a matter of exposing raw values over a network — an AI model needs meaning (units, relationships, semantics) attached to data, and a live representation it can query without touching the physical process.

15.2 Technical Description

Telemetry Envelope — no new fields; an addressed, timestamped wrapper around what `LunexObject` already exposes:

```
{
  "path"        : "lunex://.../PT-101"      (Sub-model 2)
  "timestamp"   : ISO 8601
  "class"       : "Sensor.PressureSensor"    (Sub-model 1)
  "state"       : State                       (Sub-model 4)
  "health"      : { score, flags }
  "properties"  : { value, unit, ... }
}
```

The same envelope shape applies to a Sensor, a Firewall/IDS, or an Interlock — one shape, every class.

Context Layer — semantic relationships beyond the containment chain, e.g. `measures`, `partOf`, `hasUnit`, `semanticTag`. Sub-model 2 answers "what contains what"; the Context Layer answers "what relates to what, and what does it mean."

Digital Twin — the AI-side mirror an AI model actually queries; it never touches the physical Device directly. Kept in sync by the Telemetry Envelope at whatever rate the Sub-model 3 Cloud & Analytics flag allows. The Twin holds only the **current** state — for historical state, see Sub-model 19 (Historian).

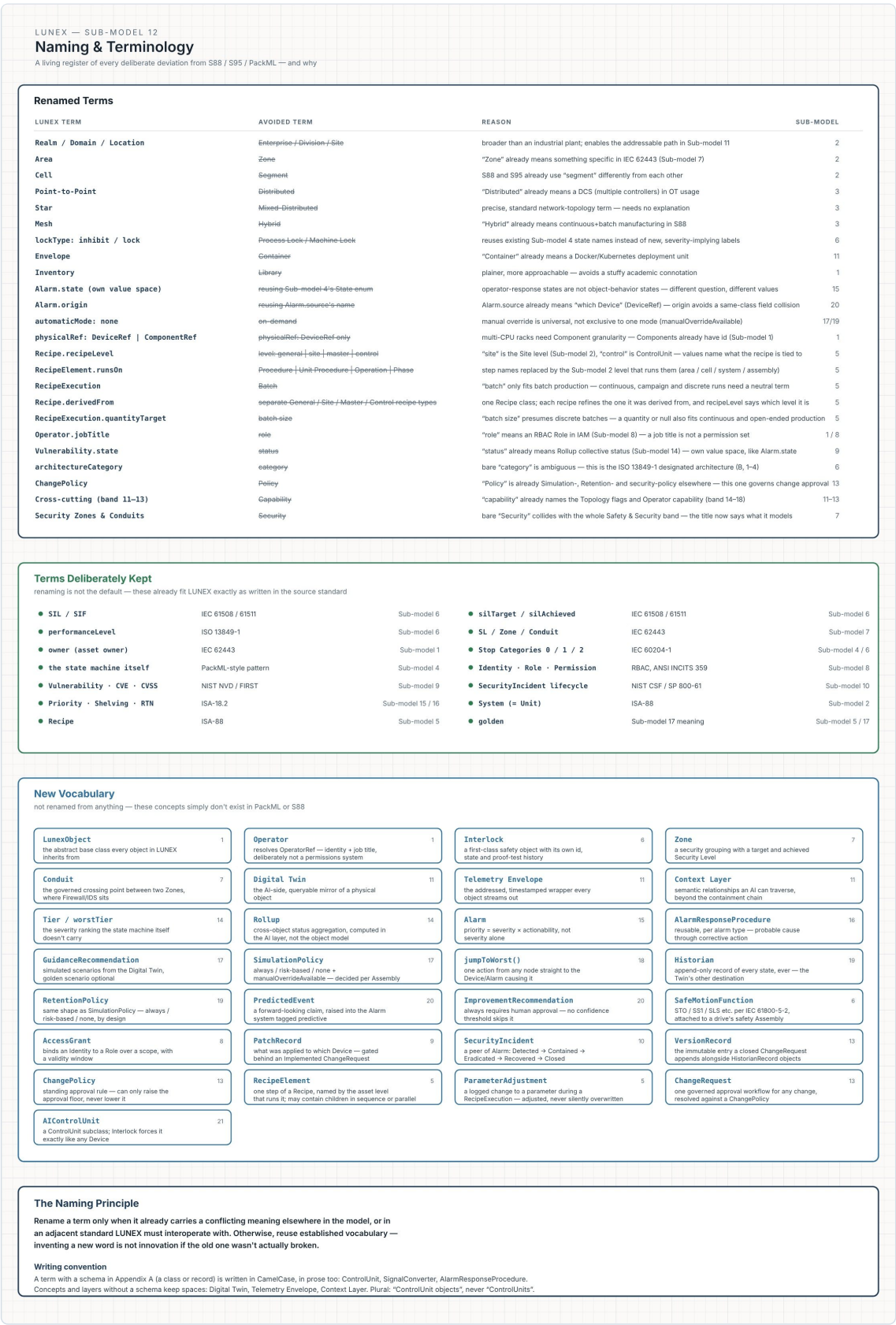
Rollup resolution note: collective/rollup status, deferred in Sub-model 4, is computed in this layer (formalized fully in Sub-model 14) — worst-case tier, tier counts, and the full contributor list, computed on the Digital Twin graph this layer maintains, never baked into `LunexObject` itself.

15.3 Application Guidance

1. Implement the Telemetry Envelope as the single outbound message shape from every ControlUnit with Cloud & Analytics enabled (Sub-model 3) — do not design a separate schema per object class.
2. Populate Context Layer relationships deliberately during commissioning; they are not inferred automatically from the containment chain.
3. Treat the Digital Twin as read/query-only from the AI side for anything that must reach the physical process — write paths back to the physical Device go through Sub-model 21's `AIControlUnit`, with its governance intact, not through the Twin directly.

16. Sub-model 12 — Naming & Terminology

Diagram: `lunex-naming-model.svg`



lunex-naming-model.svg

16.1 Purpose

A model that spans this many pre-existing standards will inevitably face naming collisions — a term two standards use differently, or a term this model itself needs for two different things. Sub-model 12 is not a diagram of a system; it is the living record of every such decision, so future contributors don't have to rediscover the reasoning or accidentally re-introduce a collision that was already resolved. This version also closes a related, narrower gap: nothing previously said what unit system a `properties` value or a `hasUnit` relationship (Sub-model 11) must actually draw from.

16.2 Technical Description

Sub-model 12 has no classes of its own. It maintains three registers:

1. **Renamed Terms** — every deliberate deviation from S88/S95/PackML, with the avoided term and the specific reason (e.g. `Realm/Domain/Location` instead of `Enterprise/Division/Site`; `Envelope` instead of `Container`, which already means a Docker/Kubernetes deployment unit; `Alarm.origin` instead of reusing `Alarm.source`'s name, which already means "which Device"; `automaticMode: none` instead of `on-demand`, which wrongly implied manual override was exclusive to one policy mode; and, from the naming review of the sub-models added in v0.2: `Operator.jobTitle` instead of `role` (`Role` is an IAM class), `Vulnerability.state` instead of `status` (`status` means Rollup), `architectureCategory` instead of `category`, `ChangePolicy` instead of `Policy`, the band name *Cross-cutting* instead of *Capability* (which names the Topology flags and the Operator capability band), Sub-model 7's title *Security Zones & Conduits* instead of *Security*, `Recipe.recipeLevel` and `RecipeElement.runsOn` instead of ISA-88's General/Site/Master/Control and Procedure/Unit Procedure/Operation/Phase, `RecipeExecution` instead of *Batch*, `Recipe.derivedFrom` instead of separate recipe types, and `quantityTarget` instead of *batch size*).
2. **Terms Deliberately Kept** — for balance, every term reused unchanged from a source standard (SIL/SIF, SL/Zone/Conduit, IEC 60204-1 stop categories, ISA-18.2's Priority/Shelving/RTN, CVE/CVSS, NIST CSF's Respond/Recover vocabulary, ISA-88's `Recipe`, and *golden* with the meaning it already has in Sub-model 17), so the model does not read as change-for-its-own-sake.
3. **New Vocabulary** — concepts that simply don't exist in PackML or S88 at all (LunexObject, Interlock, Zone, Conduit, Digital Twin, Telemetry Envelope, Context Layer, Tier/worstTier, Rollup, Alarm, AlarmResponseProcedure, GuidanceRecommendation, SimulationPolicy, jumpToWorst(), Historian, RetentionPolicy, PredictedEvent, ImprovementRecommendation, AIControlUnit, Recipe/RecipeElement/RecipeExecution/ParameterAdjustment, ChangeRequest, Identity/Role/AccessGrant, Vulnerability/PatchRecord, SecurityIncident).

The naming principle, which every entry above was tested against:

> *Rename a term only when it already carries a conflicting meaning elsewhere in the model, or in an adjacent standard LUNEX must interoperate with. Otherwise, reuse established vocabulary — inventing a new word is not innovation if the old one wasn't actually broken.*

Writing convention: a term that has a schema in Appendix A — a class or record — is written in CamelCase, in running text as well as in schemas (`ControlUnit`, `SignalConverter`, `AlarmResponseProcedure`, `PredictedEvent`). Concepts and layers that have no schema keep ordinary spacing (Digital Twin, Telemetry Envelope, Context Layer, Severity Tier). A plural is written as "ControlUnit objects", never "ControlUnits", because the plural is not itself a class. The same term is therefore always written the same way.

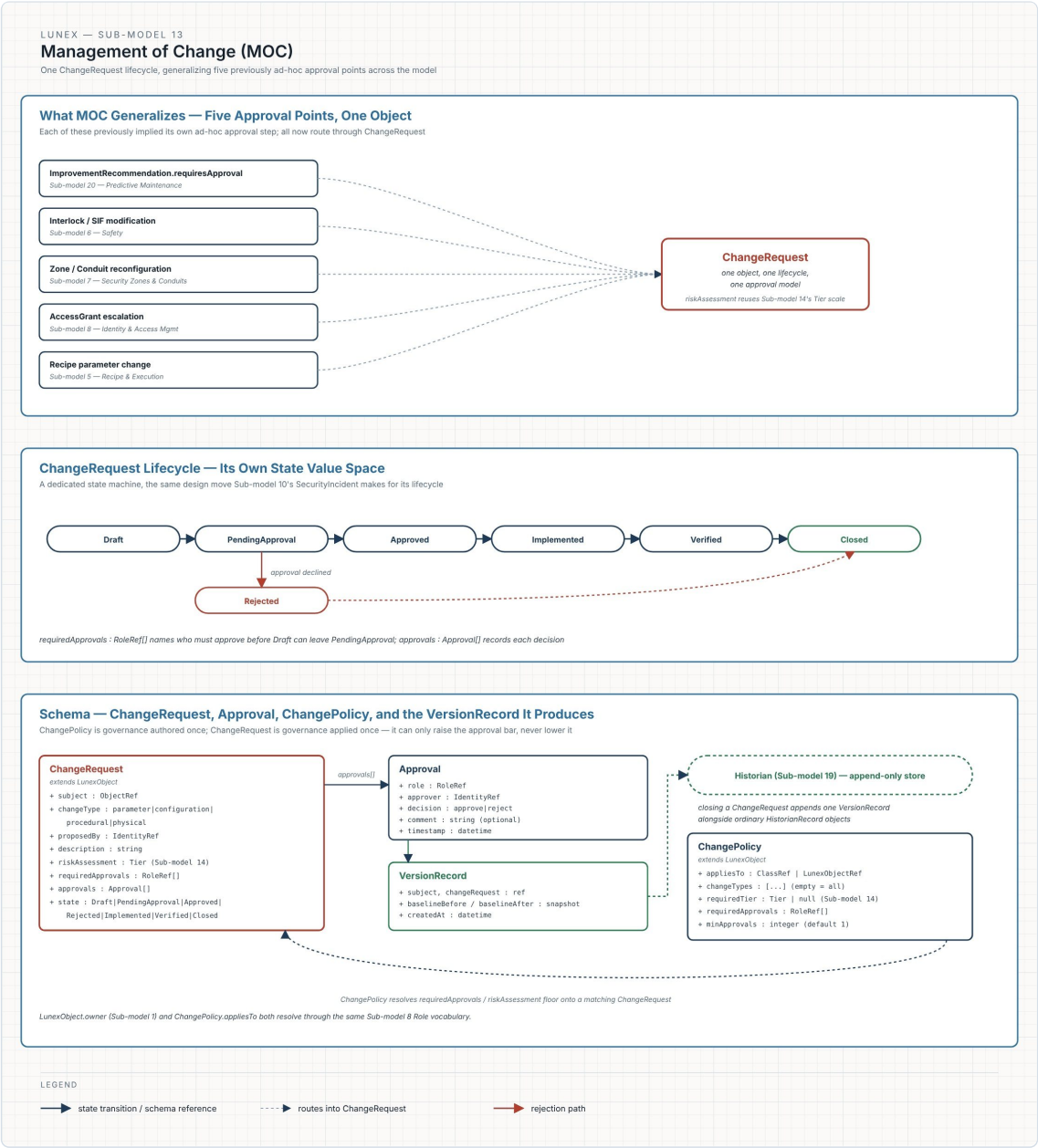
Unit-of-measure convention: every `properties` value and every `hasUnit` Context Layer relationship (Sub-model 11) must express its unit as a UCUM (Unified Code for Units of Measure) code — reused unchanged, chosen over inventing a LUNEX-specific unit list because UCUM already disambiguates the cases that actually cause plant-floor confusion (`bar` vs `bar(g)` vs `psi`, `°C` vs `K`). This is a convention, not a new class — it constrains the *value* already carried in `hasUnit`, exactly as `tag` constrains a string that already existed on `LunexObject`.

16.3 Application Guidance

1. Before introducing any new term while extending LUNEX, search this register first — the collision may already be documented, or the term you need may already exist under a different, deliberately-chosen name.
2. When a genuine collision is found (as happened between Sub-model 15's `Alarm.source` and Sub-model 20's addition), resolve it by adding an entry here, not by silently picking a name and moving on — the register's value is in being complete, not just correct.
3. Treat "Terms Deliberately Kept" as equally load-bearing as "Renamed Terms" — a proposal to rename SIL, SL, or ISA-18.2's vocabulary should be treated with the same scrutiny as a proposal to *not* rename something that actually collides.
4. Reject a `hasUnit` value at commissioning time if it is not a valid UCUM code — do not allow a free-text unit string to pass silently; it is exactly the kind of drift this sub-model exists to prevent.
5. Write every term in the form the writing convention above prescribes; a class that appears in prose as two words ("Control Unit") is a defect, not a style choice.
6. Plain-language descriptions of the sub-models (the website's one-sentence intents, the README) use a model term only where it is the real name of the thing, spelled exactly as in this register. When a term is renamed here, update those descriptions in the same change.

17. Sub-model 13 — Management of Change

Diagram: `lunex-moc-model.svg`



lunex-moc-model.svg

17.1 Purpose

Several sub-models already create something that needs approval before it takes effect: `ImprovementRecommendation.requiresApproval` (Sub-model 20), an Interlock/SIF modification (Sub-model 6), a Zone/Conduit reconfiguration (Sub-model 7), an `AccessGrant` escalation (Sub-model 8), a patch (Sub-model 9), an `equipment` / `execution`-level Recipe parameter change (Sub-model 5). Each of these has grown its own ad hoc notion of "someone approves this," with no shared workflow, no audit trail, and no record of what configuration was in effect before the change. Sub-model 13 generalizes that into one governed object, reused by all of them, rather than five parallel approval flows.

That generalization covers the workflow — *that* a change is approved, by whom, and what it changed. It does not yet cover the rule the workflow is enforcing: *why* this `changeType`, on this class of object, needs two approvers instead of one, or needs Tier 1 risk sign-off rather than Tier 3. Without a place to state that once, the rule lives only in whoever configured `requiredApprovals` on each `ChangeRequest`, individually, from memory — the same ad hoc problem MOC was built to

remove, one level up. `ChangePolicy` closes that: a standing, queryable rule a matching `ChangeRequest` resolves its approval requirements against, rather than reinventing them. This is governance in the ordinary sense — who is accountable (`LunexObject.owner`, Sub-model 1), what the rules are (`ChangePolicy`, below), and whether they were followed (`ChangeRequest` + `VersionRecord`, already defined) — assembled entirely from objects this specification already has reason to define, not a new governance layer bolted on top.

17.2 Technical Description

`ChangeRequest` (extends `LunexObject`; peer of `Interlock`, `Zone`, `Alarm`, `RecipeExecution`, etc.):

```
ChangeRequest extends LunexObject {
  subject            : LunexObjectRef
      (what is being changed — any object in the model)
  changeType         : parameter | configuration | procedural | physical
  proposedBy         : OperatorRef | GuidanceRecommendationRef
                       | ImprovementRecommendationRef | PredictedEventRef
    (a person, Sub-model 8's Identity via Operator, or any AI-origin trigger
     already in the model)
  description        : string
  riskAssessment     : Tier (0-3) | null
      (Sub-model 14 — reused, not a new risk scale)
  requiredApprovals  : RoleRef[]
      (Sub-model 8 — which Roles must approve, configurable per changeType)
  approvals          : Approval[]
  state              : Draft | PendingApproval | Approved | Rejected
                       | Implemented | Verified | Closed
  implementedAt      : timestamp | null
  verifiedAt         : timestamp | null
}

Approval {
  role       : RoleRef                                (Sub-model 8)
  approver   : IdentityRef                            (Sub-model 8)
  decision   : approved | rejected
  comment    : string
  timestamp  : timestamp
}
```

`ChangeRequest.state` is deliberately its own value space, for the same reason `Alarm.state` (Sub-model 15) is: a change-governance workflow answers a different question than object behavior or recipe execution, so it does not reuse Sub-model 4's machine. `riskAssessment` reuses Sub-model 14's Tier 0-3 scale directly — every risk judgment in LUNEX now speaks the same vocabulary, whether it comes from a process Rollup, a `SecurityIncident` (Sub-model 10), or a proposed change.

`VersionRecord` — produced automatically when a `ChangeRequest` reaches `Implemented`, closing the gap a `ChangeRequest` alone leaves: *that* a change happened is recorded, but not what configuration existed immediately before it, so there was no way to diff or roll back:

```
VersionRecord {
  subject         : LunexObjectRef
  changeRequest   : ChangeRequestRef
  baselineBefore  : snapshot
  baselineAfter   : snapshot
  createdAt       : timestamp
}
```

`VersionRecord` reuses the Historian's (Sub-model 19) append-only storage principle directly — it is a configuration snapshot rather than a telemetry value, not a new storage philosophy.

`ChangePolicy` — the standing rule a `ChangeRequest` resolves its `requiredApprovals` / `riskAssessment` against, instead of each one being configured from scratch:

```
ChangePolicy extends LunexObject {
  appliesTo          : ClassRef | LunexObjectRef
      (Sub-model 1 §5.3 — an entire class, or one instance)
  changeTypes        : (parameter | configuration | procedural | physical)[]
      (empty = all)
  requiredTier        : Tier (0-3) | null
      (Sub-model 14 — minimum riskAssessment this ChangePolicy enforces)
  requiredApprovals   : RoleRef[]                      (Sub-model 8)
  minApprovals         : integer
      (default 1 — e.g. 2 for a "two distinct approvers" rule)
  rationale           : string
}
```

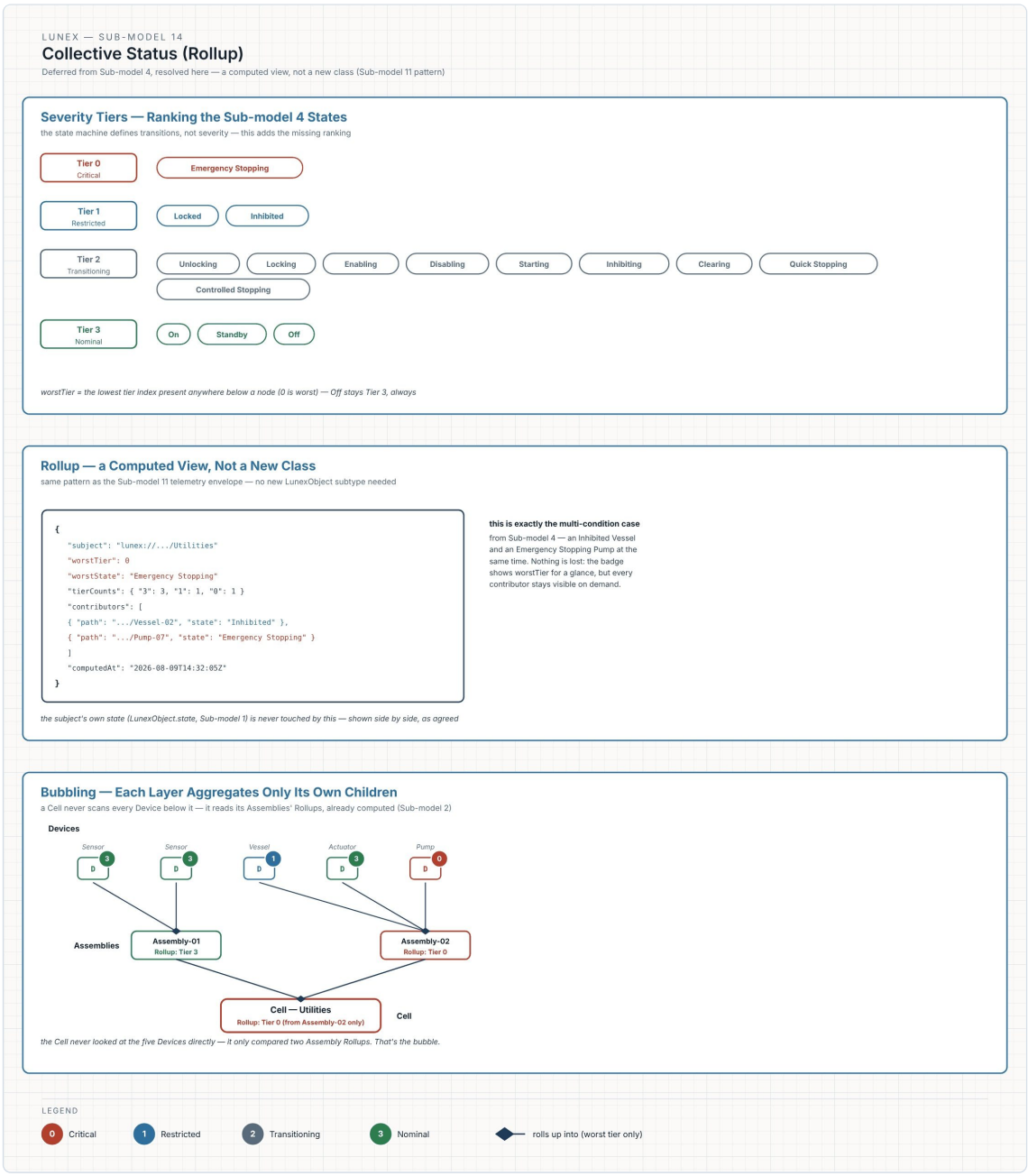
A `ChangePolicy` is governance *authored once*; a `ChangeRequest` is governance *applied once*. When a new `ChangeRequest` names a `subject` and `changeType`, the most specific matching `ChangePolicy` (an instance match beats a class match) supplies its `requiredApprovals` and the floor for its `riskAssessment` unless the `ChangeRequest` explicitly sets a stricter value — a `ChangePolicy` can only raise the bar a `ChangeRequest` must clear, never lower it. This is also where `LunexObject.owner` (Sub-model 1) earns its keep: `ChangePolicy.appliesTo` and an object's `owner` are both `Role`-shaped (via `RoleRef`/an Identity's granted Role, Sub-model 8), so "who is accountable for this object" and "what rule governs changing it" resolve through the same Sub-model 8 vocabulary rather than two unrelated ones. `ChangePolicy` does not replace `ChangeRequest.requiredApprovals` as a field — a `ChangeRequest` with no matching `ChangePolicy` still sets it directly, exactly as before; `ChangePolicy` is the optional layer that stops the same rule being re-typed on every `ChangeRequest` that needs it.

17.3 Application Guidance

1. Make `ChangeRequest` mandatory for any change to a Safety (Sub-model 6) or Zone/Conduit (Sub-model 7) object's integrity-relevant fields, any `AccessGrant` escalation (Sub-model 8), and any `recipeLevel: equipment` / `recipeLevel: execution` Recipe parameter (Sub-model 5); make it configurable, risk-based, for lower-consequence configuration changes — the same risk-based/always/none shape already used by `SimulationPolicy` / `RetentionPolicy`.
2. Set `requiredApprovals` per `changeType`, not globally — a `procedural` change to an ARP (Sub-model 16) and a `physical` change to a SIF voting architecture (Sub-model 6) do not need the same approvers.
3. Never treat `ImprovementRecommendation.requiresApproval` (Sub-model 20) as satisfied by anything other than a `ChangeRequest` reaching `Approved` — the field says approval is required, this sub-model is where it actually happens.
4. Do not skip `VersionRecord` generation to save storage on low-risk changes — the cost of a missing baseline is discovered only when a rollback is needed and the "before" state turns out not to exist.
5. Prefer one `ChangePolicy` over repeating `requiredApprovals` on every `ChangeRequest` for the same class of change — e.g. a single `ChangePolicy{ appliesTo: SIF-class, changeTypes: [physical], minApprovals: 2 }` rather than setting `minApprovals` by hand on every SIF voting-architecture `ChangeRequest`. Set `LunexObject.owner` on the objects a `ChangePolicy` targets so "who is accountable" doesn't have to be looked up separately from "what the rule is."
6. Treat `ChangeRequest` + `VersionRecord` + `SecurityIncident` (Sub-model 10) together as the audit/compliance evidence trail — who proposed a change, who approved it against which `ChangePolicy`, what configuration existed before and after, and which incidents followed. This is deliberately not a separate compliance-reporting object: a framework mapping (IEC 62443-2-1, NIST CSF) is a *view* over these three, assembled when an audit asks for it, not a fourth thing to keep in sync with them.

18. Sub-model 14 — Collective Status (Rollup)

Diagram: `lunex-rollup-model.svg`



lunex-rollup-model.svg

18.1 Purpose

Sub-model 4 deliberately left one question unanswered: if a Cell has one Device that is Inhibited and another that is Emergency Stopping at the same time, what is the Cell's status? A simple worst-case reduction loses information (which condition, where); scanning the entire tree on every query doesn't scale. Sub-model 14 answers both problems.

18.2 Technical Description

Severity Tiers — a ranking the Sub-model 4 state machine itself does not carry:

Tier	Label	States
0	Critical	Emergency Stopping
1	Restricted	Locked, Inhibited
2	Transitioning	Unlocking, Locking, Enabling, Disabling, Starting, Inhibiting, Clearing, Quick Stopping, Controlled Stopping
3	Nominal	On, Standby, Off

`Rollup` — a computed view, not a stored class:

```
Rollup {
  subject         : LunexObjectRef
  worstTier       : 0-3
  worstState      : State
  tierCounts      : { "3": n, "1": n, "0": n, ... }
  contributors    : [ {path, state}, ... ]
  computedAt      : timestamp
}
```

`worstTier` gives the at-a-glance badge. `tierCounts` and `contributors` preserve every simultaneous condition — this is what resolves the multi-condition case the state machine alone could not: an Inhibited Vessel and an Emergency Stopping Pump are both visible, not collapsed into one value.

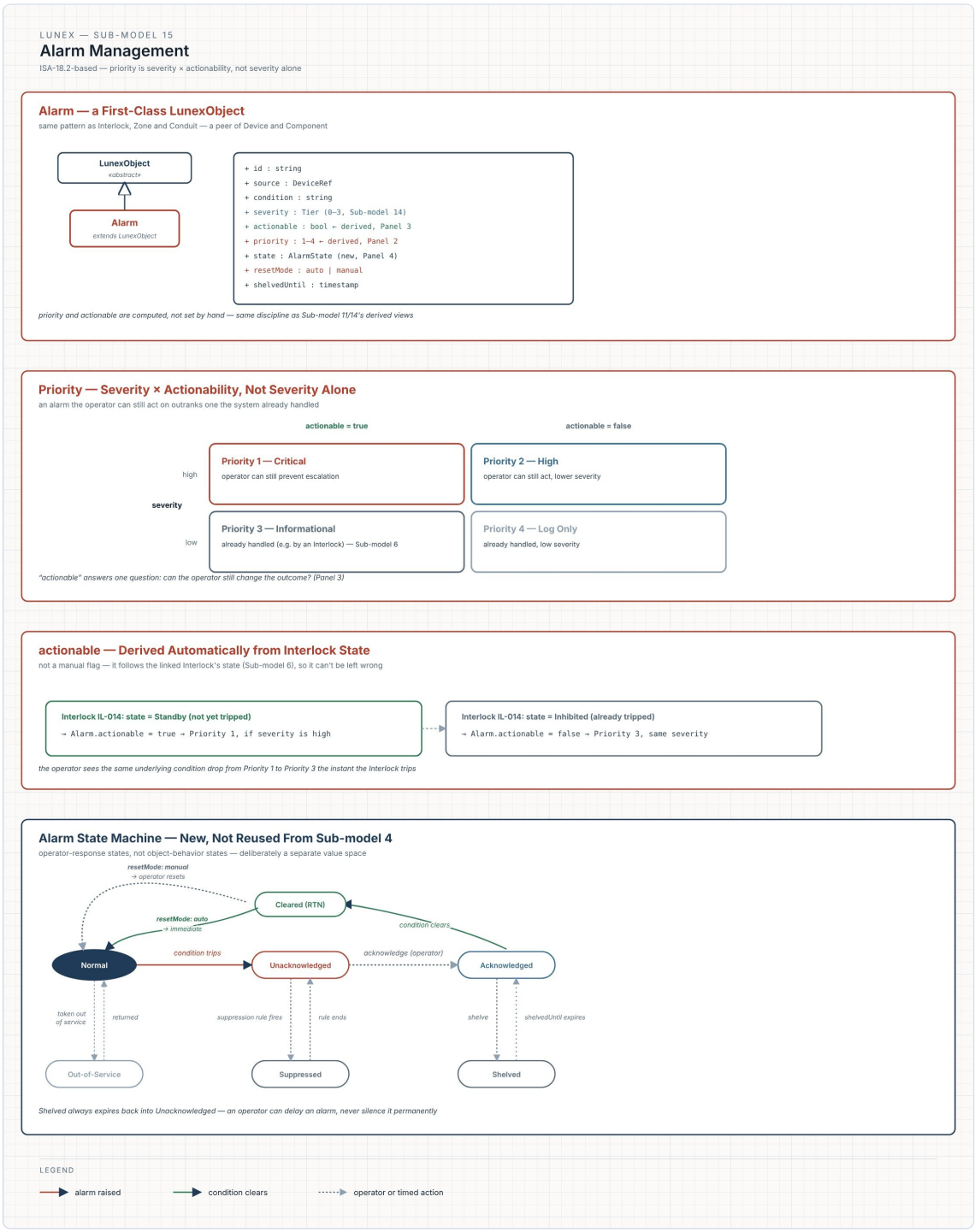
Bubbling: a parent's Rollup is computed from its direct children's Rollups only — a Cell reads its Assemblies' Rollups, not every Device beneath them. Each layer aggregates once; nothing scans the full tree.

18.3 Application Guidance

1. Compute a Rollup at every level of the Sub-model 2 hierarchy, triggered by a change in any direct child's Rollup or own state — never by polling the full subtree.
2. Surface `worstTier` as the primary visual signal (Sub-model 18); surface `tierCounts` / `contributors` on demand, not by default, to avoid overwhelming an operator screen with detail that isn't yet needed.
3. Do not conflate an object's own `state` (Sub-model 4) with its `Rollup.worstTier` — an object in nominal Standby can still have a Tier 0 Rollup because of a child, and the two values must be shown distinctly (see Sub-model 18).

19. Sub-model 15 — Alarm Management

Diagram: `lunex-alarm-model.svg`



lunex-alarm-model.svg

19.1 Purpose

ISA-18.2 defines alarm management principles but does not provide an object model. Sub-model 15 provides one, with a specific correction to a common misconception: severity alone is not priority. An alarm about a condition the system has already handled (say, an Interlock has already tripped) is *less* urgent than an alarm about the same condition while the operator could still prevent it — even though the underlying severity is identical.

19.2 Technical Description

`Alarm` (extends `LunexObject`):

```
Alarm {
  id            : string
  source        : DeviceRef
  origin        : real-time | predictive
      (Sub-model 20 — deliberately not named source, see Sub-model 12)
  condition     : string
  severity      : Tier (0-3)              (Sub-model 14)
  actionable    : bool                    (derived, see below)
  priority      : 1-4                     (derived, see below)
  state         : AlarmState              (new value space, see below)
  resetMode     : auto | manual
  shelvedUntil  : timestamp
  procedure     : AlarmResponseProcedureRef | null   (Sub-model 16)
}
```

`actionable` is derived automatically from the linked Interlock's state (Sub-model 6): `true` while the Interlock is Standby (not yet tripped), `false` once it has tripped to Inhibited/Locked. This is never a manually-set flag.

`origin` distinguishes a real-time condition from a forward-looking one; a `PredictedEvent` (Sub-model 20) raises an Alarm with `origin: predictive`, visibly tagged `PREDICTIVE`, joining the same prioritization and screen as any other alarm. `procedure` references the `AlarmResponseProcedure` (Sub-model 16) for this alarm's condition type — `null` is valid for Priority 3/4 alarms, where an ARP is optional (Sub-model 16 §20.2).

Priority matrix — severity × actionability, not severity alone:

	Actionable = true	Actionable = false
High severity	Priority 1 — Critical	Priority 3 — Informational
Low severity	Priority 2 — High	Priority 4 — Log Only

Alarm State Machine — deliberately a *new*, separate value space, not reused from Sub-model 4 (operator-response states answer a different question than object-behavior states):

States: Normal, Unacknowledged, Acknowledged, Shelved, Suppressed, Out-of-Service, Cleared (RTN).

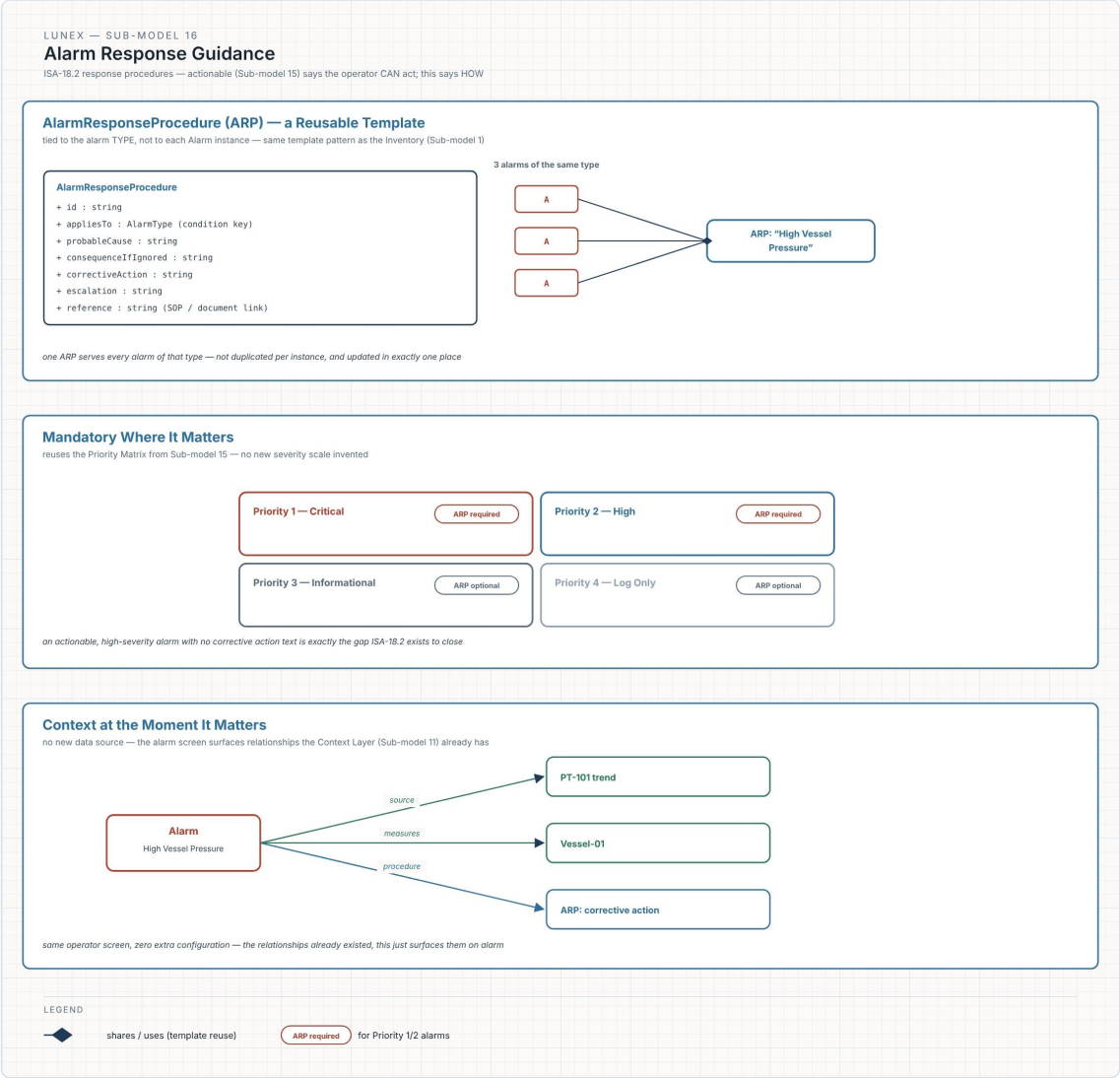
- `condition trips` → Normal → Unacknowledged (automatic)
- `acknowledge` → Unacknowledged → Acknowledged (operator action)
- `condition clears` → Acknowledged → Cleared (RTN) (automatic)
- From Cleared (RTN): if `resetMode: auto`, immediately → Normal; if `resetMode: manual`, requires an explicit `operator resets` action.
- `shelve` (operator action) → Shelved; always expires back to Unacknowledged via `shelvedUntil` — shelving can delay an alarm, never permanently silence it.
- Suppression and Out-of-Service follow the same pattern: entered and exited by explicit, timed, or rule-based actions, never a silent, permanent state.

19.3 Application Guidance

1. Set `resetMode: manual` for any alarm where an operator must consciously confirm an abnormal condition occurred, even after it self-clears (typical for safety-adjacent alarms); use `auto` for routine, low-consequence alarms.
2. Never treat `severity` alone as a proxy for `priority` in alarm rationalization work — always compute `actionable` first.
3. Configure `shelvedUntil` with a bounded, sensible expiry for every alarm class — an alarm with no expiry is a design error, not a valid configuration.

20. Sub-model 16 — Alarm Response Guidance

Diagram: `lunex-arp-model.svg`



lunex-arp-model.svg

20.1 Purpose

ISA-18.2 calls for documented response procedures, but leaves the "how" open. Sub-model 16 makes response guidance a reusable template rather than freetext duplicated per alarm instance, and ties its context-surfacing directly to what Sub-model 11 already knows — no second system to maintain.

20.2 Technical Description

`AlarmResponseProcedure` **(ARP)** — a reusable template tied to the alarm *type* (condition key), not to each Alarm instance:

```
AlarmResponseProcedure {
  id                    : string
  appliesTo             : AlarmType            (condition key)
  probableCause         : string
  consequenceIfIgnored  : string
  correctiveAction      : string
  escalation            : string
  reference              : string               (SOP / document link)
}
```

`Alarm.procedure` references one `AlarmResponseProcedure`; many Alarm instances of the same type share one ARP — it is updated once, everywhere.

Mandatory coverage, reusing the Sub-model 15 priority matrix directly: an ARP is **required** for every alarm type reachable at Priority 1 or 2; **optional** for Priority 3/4, where there is nothing left to act on.

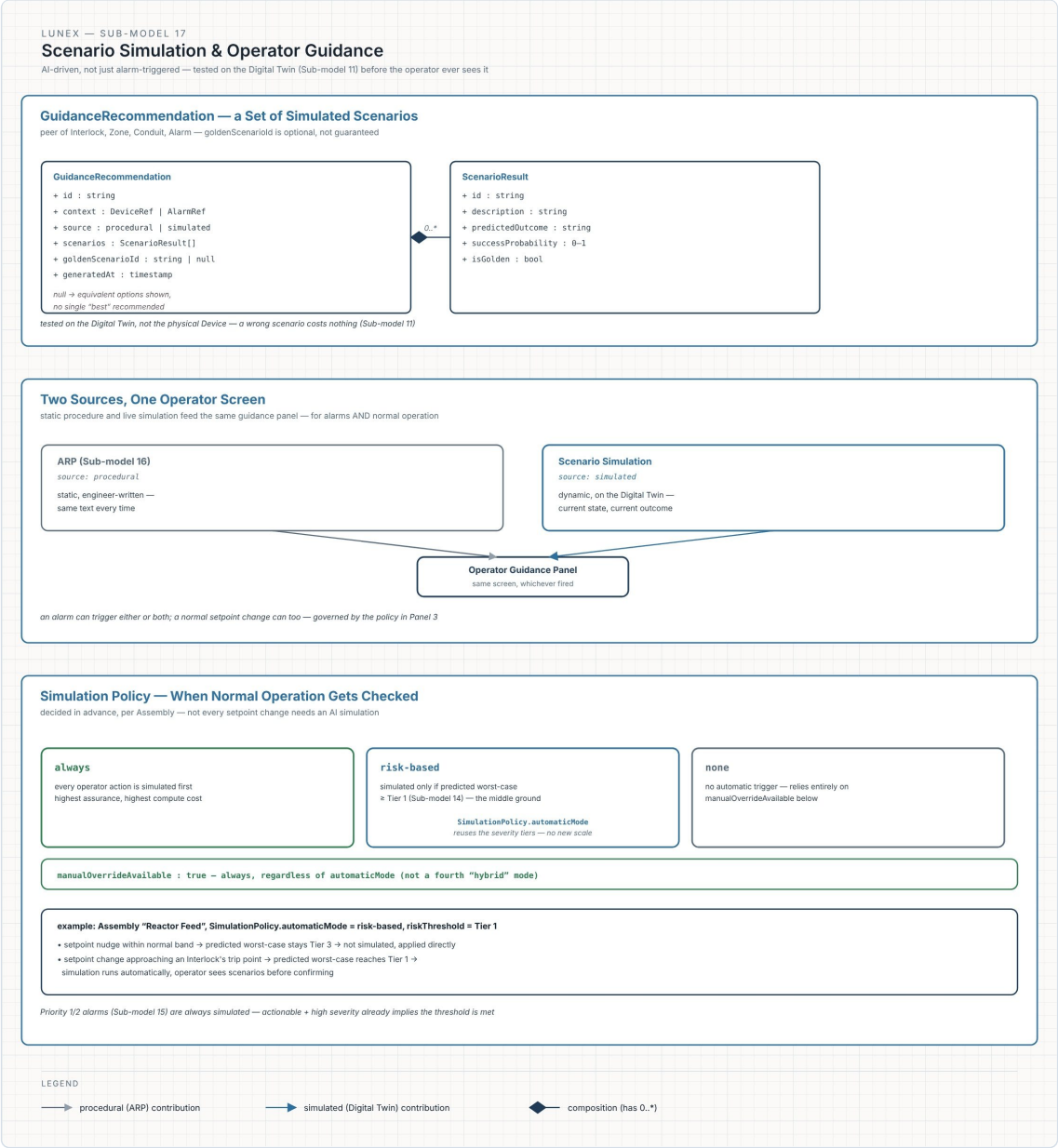
Context surfacing: when an alarm fires, the operator screen automatically surfaces the relevant Sub-model 11 Context Layer relationships (e.g. the source sensor's trend, the Vessel it `measures`) — this is not a new data source, only a new *view* over data that already exists.

20.3 Application Guidance

1. Author one ARP per distinct alarm *condition*, not per physical instrument — ten identical pressure alarms across ten vessels share one ARP.
2. Treat a missing ARP on a Priority 1/2 alarm as a configuration defect to be fixed before commissioning, not an acceptable gap to fill in later.
3. Ensure the Context Layer relationships an ARP's corrective action depends on (e.g., "check the upstream Vessel's level") are actually populated in Sub-model 11 for that object — an ARP referencing a relationship that was never configured will surface nothing useful at the moment it matters most.

21. Sub-model 17 — Scenario Simulation & Operator Guidance

Diagram: `lunex-scenario-model.svg`



lunex-scenario-model.svg

21.1 Purpose

An ARP (Sub-model 16) is static procedural knowledge, written in advance. AI makes something new possible: testing what a specific action would do, right now, on the current process state, before anyone commits to it — and not only for alarms, but for ordinary operation and configuration changes too.

21.2 Technical Description

`GuidanceRecommendation` (extends `LunexObject`; peer of Interlock, Zone, Conduit, Alarm):

```
GuidanceRecommendation {
  id                 : string
  context            : DeviceRef | AlarmRef
  source             : procedural | simulated | predictive
      (third value added in Sub-model 20)
  scenarios          : ScenarioResult[]
  goldenScenarioId   : string | null
  generatedAt        : timestamp
  state              : Open | Applied | Dismissed | Expired
}

ScenarioResult {
  id                    : string
  description           : string
  predictedOutcome      : string
  successProbability    : 0-1
  isGolden              : bool
}
```

`goldenScenarioId` may be `null` — equivalent options are a valid, honest outcome; the model does not force a single "best" recommendation when the simulation genuinely doesn't produce one.

`state` follows the same shape as `PredictedEvent` (Sub-model 20): `GuidanceRecommendation` overrides `methods()` (Sub-model 1 §5.2.1) with `apply()` and `dismiss()`, both valid only from `Open`. `Expired` is never reached by a method call — a recommendation was generated against the process state *at that moment* (Sub-model 11's Digital Twin), and once enough time passes or the underlying context has visibly moved on, it expires automatically rather than staying presentable as if it were still current.

Scenarios are tested on the Digital Twin (Sub-model 11), never the physical Device — a wrong scenario costs nothing.

`SimulationPolicy` — decided in advance, per Assembly, governing when *normal* operation (not just alarms) gets checked:

```
SimulationPolicy {
  automaticMode            : always | risk-based | none
  riskThreshold            : Tier (0-3)     (used when automaticMode: risk-based)
  manualOverrideAvailable  : true            (always, regardless of automaticMode)
}
```

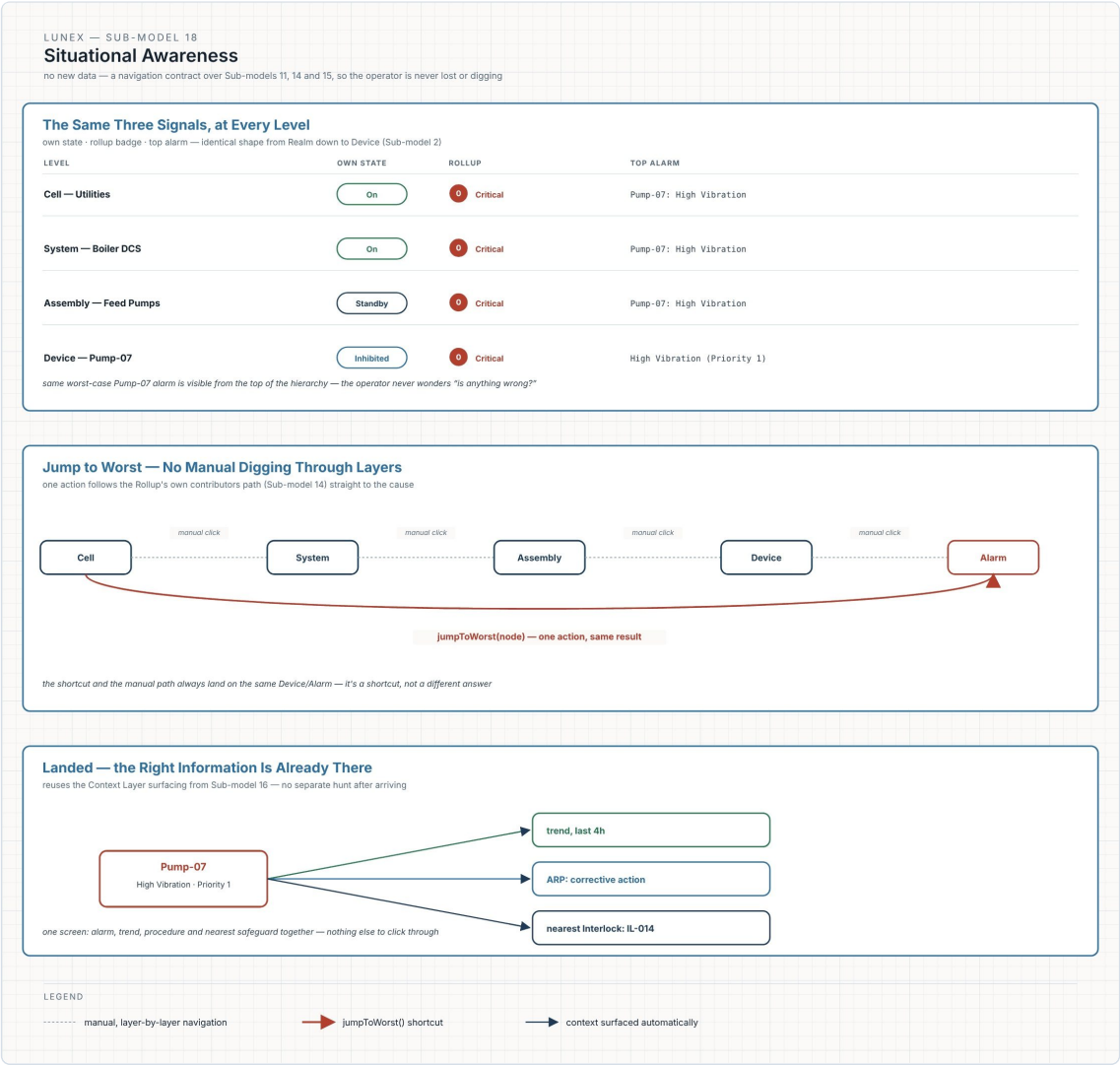
`manualOverrideAvailable` is not a fourth "hybrid" mode — manual triggering is universal and sits alongside, not instead of, the automatic modes. Priority 1/2 alarms (Sub-model 15) are always simulated, since actionable + high severity already implies the risk threshold is met.

21.3 Application Guidance

1. Set `automaticMode: risk-based` as the default for most Assemblies — `always` is reserved for the highest-consequence processes where compute cost is a non-issue; `none` for low-risk Assemblies where manual override is sufficient.
2. Never assume `goldenScenarioId` is populated in operator-facing UI — always handle the `null` case by presenting equivalent options clearly, not by arbitrarily picking one.
3. Route both `procedural` and `simulated` guidance to the same Operator Guidance Panel — do not build separate interfaces per source; the `source` field is what tells the operator which kind of claim they're looking at.

22. Sub-model 18 — Situational Awareness

Diagram: `lunex-sa-model.svg`



lunex-sa-model.svg

22.1 Purpose

Rollup (Sub-model 14), priority (Sub-model 15), and Context (Sub-model 11) each solve part of "does the operator know what's going on" — but without a navigation contract tying them together consistently at every level, an operator still has to learn a different mental model at each layer of the hierarchy, and still has to click through every intermediate level manually to find the actual cause of a problem.

22.2 Technical Description

Sub-model 18 introduces no new stored data — it is a **navigation and presentation contract** over Sub-models 11, 14, and 15.

The same three signals at every level, from Realm down to Device, identical in shape:

1. Own state (Sub-model 4)
2. Rollup badge — `worstTier` + count (Sub-model 14)
3. Top alarm — highest-priority active alarm at or below this level (Sub-model 15)

`jumpToWorst(node)` — a function, not stored data: follows a Rollup's own `contributors` path from any node directly to the Device/Alarm responsible for the worst status, in one action. It always lands on the same object the manual layer-by-layer path would reach — it is a shortcut, never a different answer.

Landed context: once navigation reaches the actual cause, the relevant Context Layer relationships and ARP are already surfaced (reusing Sub-model 16's pattern) — no further manual search is required.

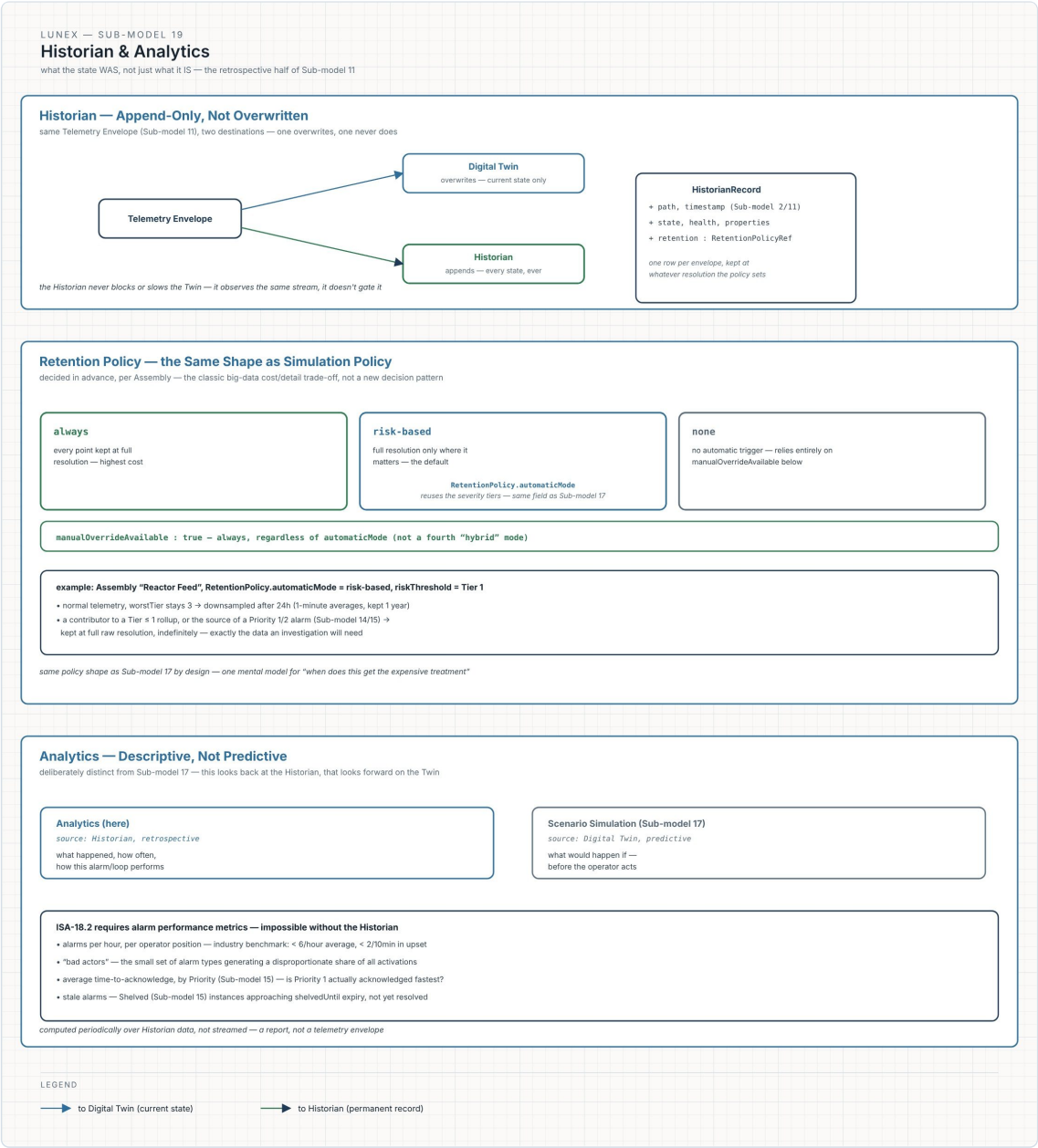
A `SecurityIncident` (Sub-model 10) surfaces through the same three-signal pattern rather than a separate security console — its `severity` Tier slots into the Rollup badge exactly as a process condition would, so an operator never has to learn a second screen to know something is wrong.

22.3 Application Guidance

1. Implement the three-signal pattern identically at every hierarchy level in the operator UI — resist the temptation to show "more detail" at lower levels and "just a summary" at higher levels using different widgets; consistency of shape is the point.
2. Implement `jumpToWorst()` as a literal traversal of `Rollup.contributors`, not a separate query — this guarantees the shortcut and the manual path can never disagree.
3. Do not add additional context-gathering logic in Sub-model 18 itself; if the right information isn't appearing on arrival, the gap is in Sub-model 11's Context Layer or Sub-model 16's ARP coverage, not in the navigation layer.

23. Sub-model 19 — Historian & Analytics

Diagram: `lunex-historian-model.svg`



lunex-historian-model.svg

23.1 Purpose

Sub-model 11 covers what the state *is*; nothing in the model up to this point covers what the state *was*. Without a historical record, ISA-18.2's required alarm performance metrics cannot be computed, no trend is available to underpin Sub-model 20's predictions, and no investigation after an incident has anything to examine.

23.2 Technical Description

Historian — an append-only record of every Telemetry Envelope (Sub-model 11) ever emitted, observing the same stream as the Digital Twin without gating or slowing it. Where the Twin overwrites (current state only), the Historian appends (every state, ever).

```
HistorianRecord {
  path, timestamp    (Sub-model 2/11)
  state, health, properties
  retention          : RetentionPolicyRef
}
```

`RetentionPolicy` — deliberately the same shape as `SimulationPolicy` (Sub-model 17): the classic big-data cost/detail trade-off, resolved with one mental model rather than a new one.

```
RetentionPolicy {
  automaticMode            : always | risk-based | none
  riskThreshold            : Tier (0-3)
  manualOverrideAvailable  : true
}
```

Example: with `automaticMode: risk-based` and `riskThreshold: Tier 1` — normal telemetry (worstTier stays 3) is downsampled after 24h to 1-minute averages, kept 1 year; a contributor to a Tier ≤1 Rollup, or the source of a Priority 1/2 alarm, is kept at full raw resolution indefinitely.

Analytics — deliberately descriptive/retrospective, contrasted with Sub-model 17's predictive simulation: what happened, how often, how a given alarm or loop performs, computed periodically over Historian data (a report, not a telemetry stream).

ISA-18.2 alarm performance metrics, computed here because they are impossible without the Historian: - Alarms per hour, per operator position (industry benchmark: <6/hour average, <2/10min during a process upset) - "Bad actors" — the small set of alarm types generating a disproportionate share of all activations - Average time-to-acknowledge, by Priority (Sub-model 15) - Stale alarms — Shelved instances approaching `shelvedUntil` expiry, still unresolved

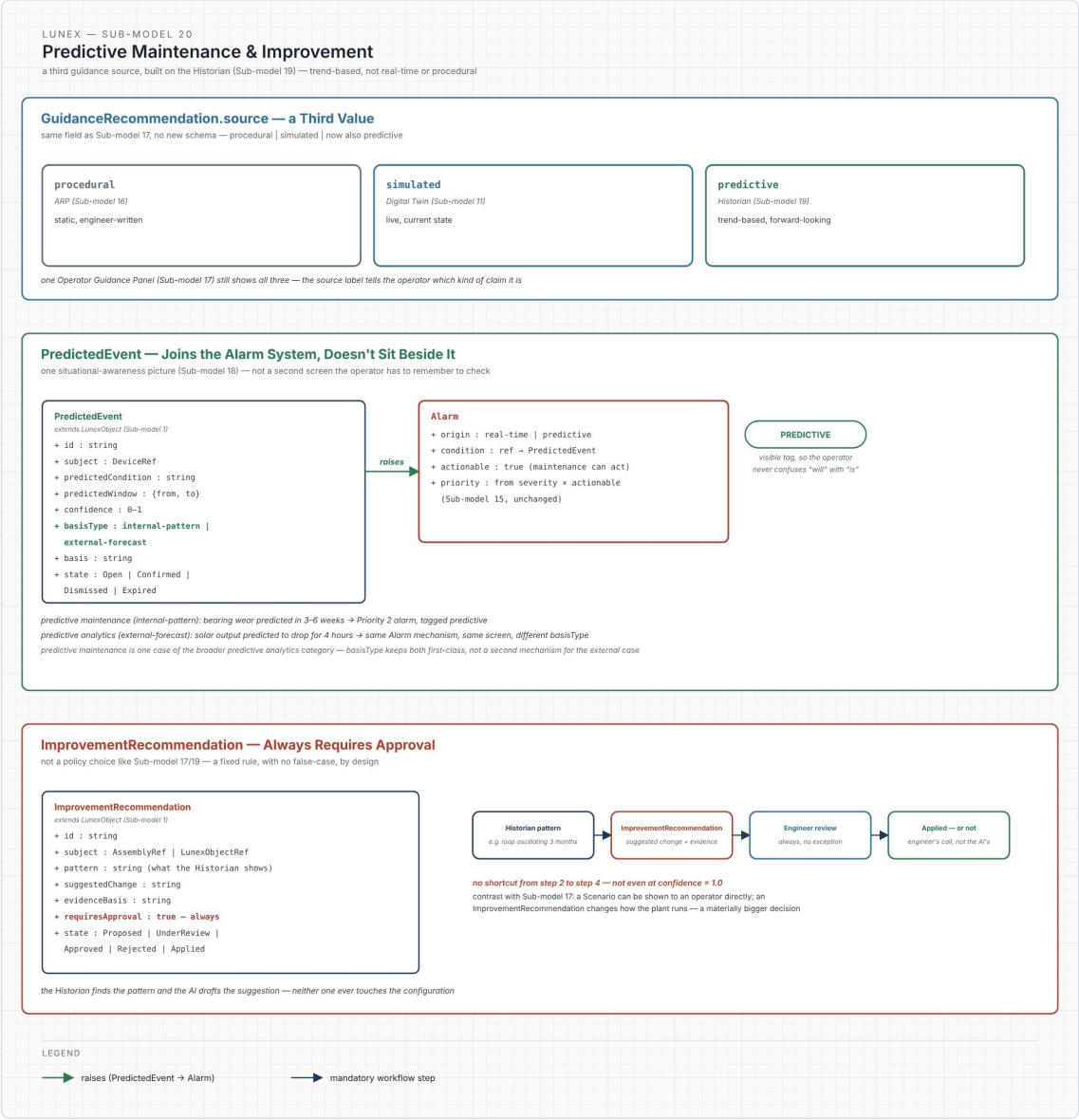
`VersionRecord` baselines (Sub-model 13) are stored alongside `HistorianRecord`s, reusing the same append-only store — a configuration history and a telemetry history are the same storage concern with two different payload shapes, not two separate systems.

23.3 Application Guidance

1. Set `RetentionPolicy` per Assembly using the same risk-based defaults recommended for `SimulationPolicy` (Sub-model 17) — the two should usually be configured together, not independently.
2. Run the ISA-18.2 performance metrics on a fixed schedule (e.g., weekly/monthly), not on demand only — trend visibility over time is the point.
3. Never write Analytics output back into the Digital Twin — it is a separate, retrospective product, consumed by engineers and Sub-model 20, not by the real-time operator picture.

24. Sub-model 20 — Predictive Maintenance & Improvement

Diagram: `lunex-predictive-model.svg`



lunex-predictive-model.svg

24.1 Purpose

Once a Historian exists, trend-based prediction becomes possible — not just "what is happening" or "what would happen if," but "what is likely to happen, unprompted." Trend extrapolation from internal history is one source of that — the original motivation for this sub-model — but not the only one: a forecast can equally come from outside the plant entirely (a weather forecast driving an anticipatory setpoint change, an energy-price forecast driving a load-shift). Both are handled by the same object, distinguished by where the forecast actually comes from. This introduces two distinct needs: surfacing a prediction to an operator without creating a second alarm-like system to monitor, and turning a detected pattern into a suggested configuration change without ever letting AI confidence substitute for human judgment.

24.2 Technical Description

`GuidanceRecommendation.source` gains a third value: `predictive`, built on the Historian (Sub-model 19) rather than the live Digital Twin or a static procedure.

`PredictedEvent` (extends `LunexObject`; peer of Interlock, Alarm, etc.):

```
PredictedEvent {
  id                    : string
  subject               : DeviceRef
  predictedCondition     : string
  predictedWindow        : { from, to }
  confidence              : 0-1
  basisType               : internal-pattern | external-forecast
  basis                   : string
      (Historian pattern description, or the external source itself)
  state                    : Open | Confirmed | Dismissed | Expired
}
```

`basisType` distinguishes **predictive maintenance** — `internal-pattern`, extrapolated from the Historian, e.g. "bearing wear predicted in 3–4 weeks" — from the broader category of **predictive analytics**: `external-forecast`, driven by a source outside the plant entirely, e.g. "solar output predicted to drop for the next 4 hours" (weather forecast) or "load shift recommended overnight" (energy-price forecast). Predictive maintenance is a subset of predictive analytics, not a synonym for it — this sub-model's name reflects the case it was originally built for; `basisType` makes the broader case equally first-class rather than requiring a second mechanism. An `external-forecast` `PredictedEvent` does not require a Historian at all — only `internal-pattern` does.

A `PredictedEvent` **raises** into the ordinary Sub-model 15 `Alarm` system — `Alarm.origin: predictive`, visibly tagged `PREDICTIVE` — rather than occupying a separate maintenance-planning screen the operator must remember to check separately (this joins Sub-model 18's single situational-awareness picture, it doesn't sit beside it). `Alarm.origin` is deliberately not named `Alarm.source`, since that field already means "which Device" on the same class (see Sub-model 12).

`state: Expired` matters specifically for Sub-model 19's Analytics: a prediction whose window passed without confirmation is exactly the data needed to evaluate how reliable predictions actually are — for both `basisType` values equally. `PredictedEvent` overrides `methods()` (Sub-model 1 §5.2.1) with `confirm()` and `dismiss()`, both only valid from `Open`; `Expired` is never reached by a method call, only by the window passing unconfirmed.

`ImprovementRecommendation` (extends `LunexObject`):

```
ImprovementRecommendation {
  id                  : string
  subject             : AssemblyRef | LunexObjectRef
  pattern             : string             (what the Historian shows)
  suggestedChange     : string
  evidenceBasis       : string
  requiresApproval    : true                (always — no false case)
  state               : Proposed | UnderReview | Approved | Rejected | Applied
}
```

`ImprovementRecommendation` overrides `methods()` with `startReview()`, `approve()` / `reject()`, and `apply()` — one method per arrow in the workflow below, with no method that skips a step. Approval itself is routed through a `ChangeRequest` (Sub-model 13) — this sub-model's workflow decides *whether* a change is proposed and reviewed by an engineer; Sub-model 13 is where that review is actually governed and audited.

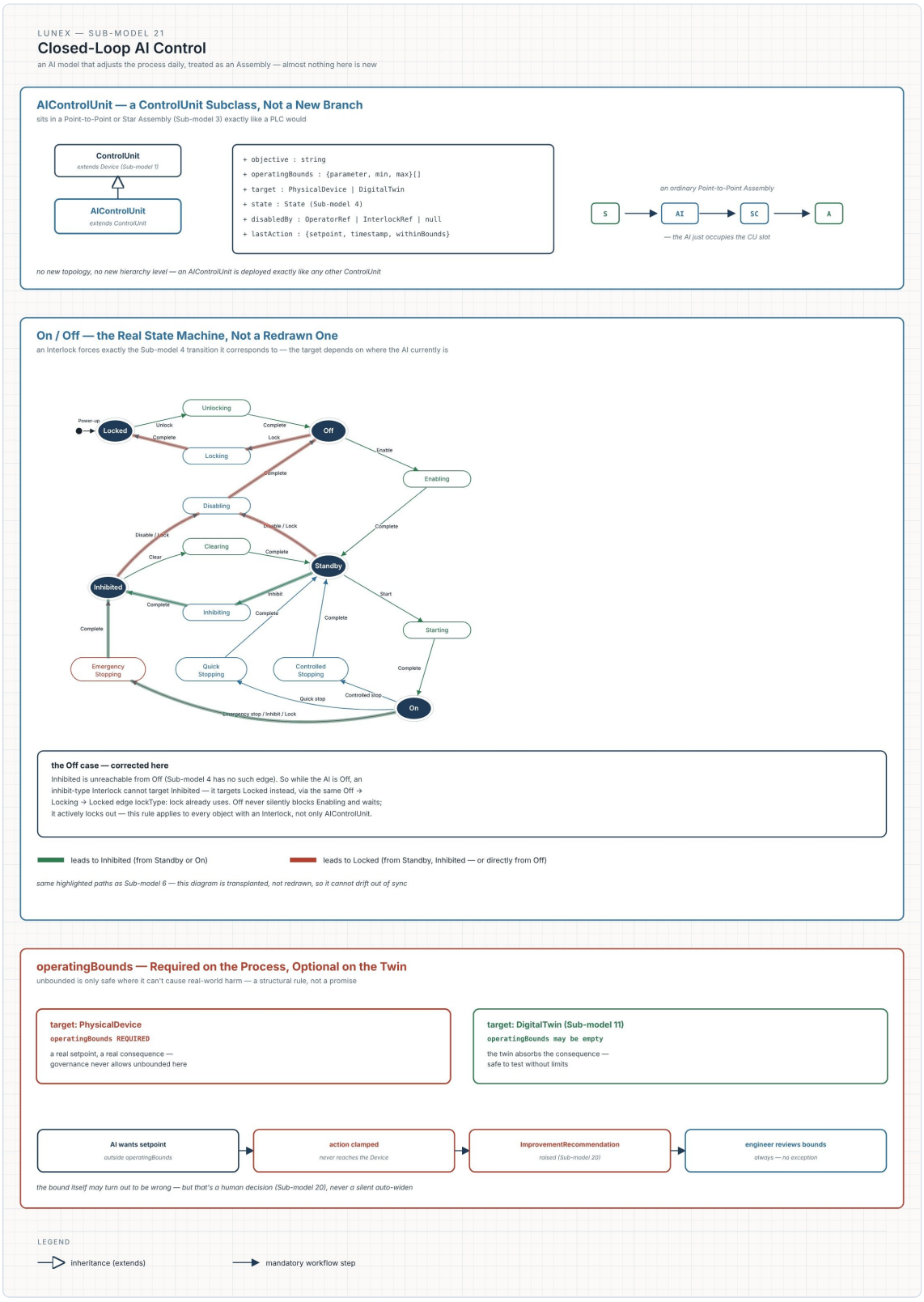
Mandatory workflow, no shortcut: Historian pattern → `ImprovementRecommendation` drafted → engineer review (always, no exception) → Applied or not, at the engineer's discretion — never the AI's. This holds even at `confidence: 1.0`. The distinction from Sub-model 17: a `ScenarioResult` can be shown to an operator directly because it only informs one decision in the moment; an `ImprovementRecommendation` changes how the plant runs going forward — a materially bigger decision that always requires human approval.

24.3 Application Guidance

1. Route every `PredictedEvent` through the same Alarm prioritization logic as real-time alarms (Sub-model 15) — do not build a separate priority scheme for predictions.
2. Never expose a mechanism to auto-apply an `ImprovementRecommendation`, regardless of confidence — `requiresApproval` is a fixed rule in this specification, not a configurable default.
3. Use `PredictedEvent.state: Expired` data as direct input to a running accuracy assessment of the prediction model generating them — a model that is frequently wrong should be visible as such, not silently trusted.

25. Sub-model 21 — Closed-Loop AI Control

Diagram: `lunex-ai-control-model.svg`



lunex-ai-control-model.svg

25.1 Purpose

Some AI models don't just recommend — they adjust the process directly, continuously, without a human in the loop for every action. This is qualitatively different from Sub-model 20's `ImprovementRecommendation` (a one-time, approved configuration change) and needs its own governance: the ability to enable/disable the AI like any other Device, and a hard boundary on what it is allowed to do autonomously.

25.2 Technical Description

`AIControlUnit` (extends `ControlUnit`, Sub-model 1 — not a new branch):

```
AIControlUnit extends ControlUnit {
  objective         : string
  operatingBounds   : { parameter, min, max }[]
  target            : PhysicalDevice | DigitalTwin
  state             : State                          (Sub-model 4, unchanged)
  disabledBy        : OperatorRef | InterlockRef | null
  lastAction        : { setpoint, timestamp, withinBounds }
}
```

`target` here means something different from `Interlock.target` (Sub-model 6) — an Interlock's `target` is the Device it forces a transition on; an AIControlUnit's `target` is what its output is routed to. The two never collide in practice, since no object is ever both classes at once; this is the same pattern already established by `state`, which means something different on `Alarm` than on most other classes (Sub-model 12). `disabledBy` resolves `OperatorRef` to a first-class `Operator` (Sub-model 1 §5.2).

It occupies an ordinary ControlUnit slot within a Point-to-Point or Star Assembly (Sub-model 3) — no new topology shape is required.

On/Off reuses the Sub-model 4 state machine, unmodified. An Interlock (Sub-model 6) forces an `AIControlUnit` to Standby or Locked exactly as it would any other Device — including the corrected Off→Locked behavior (§10.4): from Off, an inhibit-type Interlock targets Locked directly, not Inhibited. Consequently, `AIControlUnit` does **not** override `methods()` (Sub-model 1 §5.2.1): `enable()` / `disable()` / `start()` / `emergencyStop()` and the rest are exactly the standard Sub-model 4 list, unlike `Alarm`, `Interlock`, `GuidanceRecommendation`, `PredictedEvent`, or `ImprovementRecommendation`, each of which defines its own trigger set. An AI performing closed-loop control is switched on and off the same way any ControlUnit is.

`operatingBounds` governance:

`target`	`operatingBounds`
PhysicalDevice	**Required.** A real setpoint has a real consequence; governance never allows unbounded operation here.
DigitalTwin (Sub-model 11)	**May be empty.** The Twin absorbs the consequence — safe to test without limits.

An action outside `operatingBounds` is **clamped** — it never reaches the physical Device — and separately **raises an `ImprovementRecommendation`** (Sub-model 20) suggesting the bound itself be reviewed. The bound may turn out to be wrong, but that is always a human decision, never a silent auto-widening.

25.3 Application Guidance

1. Never deploy an `AIControlUnit` with `target: PhysicalDevice` and empty `operatingBounds` — this specification treats that configuration as invalid, not merely discouraged.
2. Use `target: DigitalTwin` for all model testing and tuning before any production deployment against a physical target.
3. Wire at least one Interlock to every production `AIControlUnit`, exactly as you would for any other ControlUnit performing a comparable function — an AI performing closed-loop control is not exempt from the safety model.
4. Treat repeated out-of-bounds attempts (visible via `ImprovementRecommendation` frequency) as a signal to review whether the objective or the bounds are miscalibrated — not as a reason to widen the bounds reflexively.

26. References

- ISA-88 (IEC 61512) — Batch Control
- ISA-95 (IEC 62264) — Enterprise-Control System Integration
- ISA-TR88.00.02 (PackML) — Machine and Unit States
- IEC 61508 — Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems
- IEC 61511 — Functional Safety: Safety Instrumented Systems for the Process Industry Sector
- ISO 13849-1 — Safety of Machinery: Safety-Related Parts of Control Systems (Performance Level)
- IEC 62061 — Safety of Machinery: Functional Safety of Electrical Control Systems (SIL for machinery)
- IEC 61800-5-2 — Adjustable Speed Electrical Power Drive Systems: Safety Requirements (Safe Torque Off and related safe-motion functions)
- IEC 62443 — Industrial Communication Networks: Network and System Security (including -2-1 incident response, -2-3 patch management, -3-3 system security requirements, -4-1 secure product development)
- ISA-18.2 — Management of Alarm Systems for the Process Industries
- IEC 60204-1 — Safety of Machinery: Electrical Equipment of Machines
- NIST Cybersecurity Framework (CSF) — Respond/Recover functions
- CVE (Common Vulnerabilities and Exposures) / CVSS (Common Vulnerability Scoring System)
- EU Cyber Resilience Act — vulnerability handling and SBOM requirements for connected products
- UCUM (Unified Code for Units of Measure)
- Purdue Enterprise Reference Architecture (PERA)

LUNEX is an independent work and is not affiliated with, endorsed by, or a replacement for any of the standards bodies listed above.

This document is part of the LUNEX project. See `README.md` and `LICENSE` in the repository root, and lunex.cloud.

Appendix A — Schema Reference

Every class defined across the twenty-one sub-models, collected here for lookup without paging back through each chapter. Attributes shown are exactly as specified in the relevant Sub-model section; this appendix adds nothing new; it only re-collects.

Sub-model 1 — Object / Class Model

```
LunexObject {                          (abstract base — every object has this)
  id            : string
  tag           : string
  class         : Class
  parent        : LunexObject
  state         : State                 (Sub-model 4)
  health        : Health                { score: 0-100, flags: string[] }
  coordinates   : { lat, lon } | null
  properties    : Map<Key, Value>
  owner         : RoleRef | null
      (Sub-model 8 — accountable Role, not acting Identity)
  methods()     : Procedure[]           (derived from state — Sub-model 1 §5.2.1)
}

Device extends LunexObject {           (abstract)
  physicalRef : DeviceRef | ComponentRef | null
}

Component extends LunexObject {}       (abstract — not independently addressable)

Sensor, Transducer, ControlUnit, SignalConverter, Actuator
  all extend Device, no additional attributes — distinguished by
  function-interface (Sensing / Converting / Controlling / Actuating)

Operator extends LunexObject {
  jobTitle : string   (e.g. "Control Room Operator", "Safety Engineer")
}
```

Sub-model 5 — Recipe & Execution Model

```
Recipe {
  recipeLevel : product | location | equipment | execution
  derivedFrom : RecipeRef | null
  appliesTo   : LocationRef | AreaRef | CellRef | SystemRef | null
  basedOnExecution: RecipeExecutionRef | null
  goldenExecution : RecipeExecutionRef | null
  structure   : RecipeElementRef
  parameters  : ParameterSpec[]
  version     : string
}

RecipeElement {
  id             : string
  runsOn         : area | cell | system | assembly
  target         : AreaRef | CellRef | SystemRef | AssemblyRef | null
  equipmentClass : ClassRef | null
  children       : RecipeElementRef[]
  childOrder     : sequence | parallel
}

RecipeExecution extends LunexObject {
  recipe           : RecipeRef
  equipment        : AreaRef | CellRef | SystemRef
  currentSteps     : RecipeElementRef[]
  parameterValues  : Map<string, value>
  adjustments      : ParameterAdjustment[]
  startTime        : timestamp
  endTime          : timestamp | null
  quantityTarget   : quantity | null
  genealogy        : RecipeExecutionRef[]
  state            : State
      (Sub-model 4 — reused unchanged, no override)
}
```

Sub-model 6 — Safety

```
Interlock extends LunexObject {
  condition          : DeviceRef
  action             : force | block
  target             : DeviceRef
  lockType           : inhibit | lock
  safetyStandard     : IEC61508-SIL | ISO13849-PL | IEC62061-SIL
  performanceLevel   : a | b | c | d | e | null
  architectureCategory : B | 1 | 2 | 3 | 4 | null
  proofTestHistory   : ProofTest[]
}

SIF Assembly {
  independent          : true
  silTarget            : SIL (1-4)
  silAchieved          : SIL (1-4)
  pfdAvg               : number | null
  pfh                  : number | null
  votingArchitecture    : { sensor: MooN, logicSolver: MooN, finalElement: MooN }
}

SafeMotionFunction {
  device              : DeviceRef
  supportedFunctions  : (STO | SS1 | SS2 | SOS | SLS | SDI | SLP)[]
  stoCategory         : Category (0-2)
}
```

Sub-model 7 — Security Zones & Conduits

```
Zone extends LunexObject {
  members                 : LunexObjectRef[]
  securityLevelTarget     : SL-T (0-4)
  securityLevelAchieved   : SL-A (0-4)
  purdueLevel             : ref               (Sub-model 2)
}

Conduit extends LunexObject {
  zoneA                   : ZoneRef
  zoneB                   : ZoneRef
  controlUnit             : DeviceRef          (Firewall / IDS)
  securityLevelTarget     : SL-T (0-4)
}
```

Sub-model 8 — Identity & Access Management

```
Identity extends LunexObject {
  identityType   : human | service-account
  credentialRef  : string
  operator       : OperatorRef | null
}

Role {
  permissions  : PermissionRef[]
}

Permission {
  action  : view | operate | configure | approve
  scope   : LunexObjectRef | ClassRef
}

AccessGrant {
  identity    : IdentityRef
  role        : RoleRef
  scope       : LunexObjectRef
  grantedAt   : timestamp
  expiresAt   : timestamp | null
}
```

Sub-model 9 — Vulnerability & Patch Management

```
Vulnerability {
  cveRef           : string
  cvss             : number
  affectedDevices  : DeviceRef[]
  state            : open | mitigated | patched | accepted-risk   (own value space)
}

PatchRecord {
  device       : DeviceRef
  version      : string
  appliedAt    : timestamp
  verifiedAt   : timestamp | null
}
```

Sub-model 10 — Security Incident Response

```
SecurityIncident extends LunexObject {
  detectedVia   : ZoneRef | ConduitRef | DeviceRef
  severity      : Tier (0-3)
  relatedChange : ChangeRequestRef | null
  state         : Detected | Contained | Eradicated | Recovered | Closed
      (own value space)
}
```

Sub-model 13 — Management of Change

```
ChangeRequest extends LunexObject {
  subject            : LunexObjectRef
  changeType         : parameter | configuration | procedural | physical
  proposedBy         : OperatorRef | GuidanceRecommendationRef
                       | ImprovementRecommendationRef | PredictedEventRef
  description        : string
  riskAssessment     : Tier (0-3) | null
  requiredApprovals  : RoleRef[]
  approvals          : Approval[]
  state              : Draft | PendingApproval | Approved | Rejected
                       | Implemented | Verified | Closed
      (own value space)
  implementedAt      : timestamp | null
  verifiedAt         : timestamp | null
}

Approval {
  role       : RoleRef
  approver   : IdentityRef
  decision   : approved | rejected
  comment    : string
  timestamp  : timestamp
}

VersionRecord {
  subject         : LunexObjectRef
  changeRequest   : ChangeRequestRef
  baselineBefore  : snapshot
  baselineAfter   : snapshot
  createdAt       : timestamp
}

ChangePolicy extends LunexObject {
  appliesTo          : ClassRef | LunexObjectRef
  changeTypes        : (parameter | configuration | procedural | physical)[]
      (empty = all)
  requiredTier       : Tier (0-3) | null
  requiredApprovals  : RoleRef[]
  minApprovals       : integer            (default 1)
  rationale          : string
}
```

Sub-model 14 — Collective Status

```
Rollup {                              (computed view, not a stored class)
  subject         : LunexObjectRef
  worstTier       : 0-3
  worstState      : State
  tierCounts      : { "3": n, "1": n, "0": n, ... }
  contributors    : [ {path, state}, ... ]
  computedAt      : timestamp
}
```

Sub-model 15 — Alarm Management

```
Alarm extends LunexObject {
  source        : DeviceRef
  origin        : real-time | predictive  (Sub-model 20)
  condition     : string
  severity      : Tier (0-3)              (Sub-model 14)
  actionable    : bool                    (derived from linked Interlock's state)
  priority      : 1-4                     (derived: severity × actionable)
  state         : AlarmState              (own value space — Sub-model 15 §19.2)
  resetMode     : auto | manual
  shelvedUntil  : timestamp
  procedure     : AlarmResponseProcedureRef | null   (Sub-model 16)
}
```

Sub-model 16 — Alarm Response Guidance

```
AlarmResponseProcedure {
  appliesTo               : AlarmType            (condition key)
  probableCause           : string
  consequenceIfIgnored    : string
  correctiveAction        : string
  escalation              : string
  reference                : string               (SOP / document link)
}
```

Sub-model 17 — Scenario Simulation & Operator Guidance

```
GuidanceRecommendation extends LunexObject {
  context            : DeviceRef | AlarmRef
  source             : procedural | simulated | predictive
  scenarios          : ScenarioResult[]
  goldenScenarioId   : string | null
  generatedAt        : timestamp
  state              : Open | Applied | Dismissed | Expired
}

ScenarioResult {
  description           : string
  predictedOutcome      : string
  successProbability    : 0-1
  isGolden              : bool
}

SimulationPolicy {
  automaticMode            : always | risk-based | none
  riskThreshold            : Tier (0-3)     (used when automaticMode: risk-based)
  manualOverrideAvailable  : true            (always)
}
```

Sub-model 19 — Historian & Analytics

```
HistorianRecord {
  path, timestamp    (Sub-model 2/11)
  state, health, properties
  retention          : RetentionPolicyRef
}

RetentionPolicy {
  automaticMode            : always | risk-based | none
  riskThreshold            : Tier (0-3)
  manualOverrideAvailable  : true
}
```

Sub-model 20 — Predictive Maintenance & Improvement

```
PredictedEvent extends LunexObject {
  subject               : DeviceRef
  predictedCondition     : string
  predictedWindow        : { from, to }
  confidence              : 0-1
  basisType               : internal-pattern | external-forecast
  basis                   : string
      (Historian pattern, or the external source)
  state                    : Open | Confirmed | Dismissed | Expired
}

ImprovementRecommendation extends LunexObject {
  subject             : AssemblyRef | LunexObjectRef
  pattern             : string             (what the Historian shows)
  suggestedChange     : string
  evidenceBasis       : string
  requiresApproval    : true                (always — no false case)
  state               : Proposed | UnderReview | Approved | Rejected | Applied
}
```

Sub-model 21 — Closed-Loop AI Control

```
AIControlUnit extends ControlUnit {
  objective         : string
  operatingBounds   : { parameter, min, max }[]
  target            : PhysicalDevice | DigitalTwin
  disabledBy        : OperatorRef | InterlockRef | null
  lastAction        : { setpoint, timestamp, withinBounds }
}
```

`methods()` **quick reference** (Sub-model 1 §5.2.1)

State	Triggers
`Locked`	`unlock()`
`Off`	`lock()`, `enable()`
`Standby`	`start()`, `disable()`, `inhibit()`
`Inhibited`	`clear()`, `disable()`
`On`	`controlledStop()`, `quickStop()`, `emergencyStop()`
any transient state	*(empty)*
`Alarm` (override)	`acknowledge()`, `shelve()`, `operatorReset()`, `suppress()` / `unsuppress()`, `takeOutOfService()` / `returnToService()`
`Interlock` (override)	`clear()`
`GuidanceRecommendation` (override)	`apply()`, `dismiss()`
`PredictedEvent` (override)	`confirm()`, `dismiss()`
`ImprovementRecommendation` (override)	`startReview()`, `approve()`, `reject()`, `apply()`
`RecipeExecution`	*(no override — reuses the standard Sub-model 4 list)*
`ChangeRequest` (override)	`submit()`, `approve()`, `reject()`, `implement()`, `verify()`, `close()`
`SecurityIncident` (override)	`detect()`, `contain()`, `eradicate()`, `recover()`, `close()`